Computers and Electrical Engineering 132 (2026) 110999

Contents lists available at ScienceDirect

Computers and Electrical Engineering

journal homepage: www.elsevier.com/locate/compeleceng

BMGANet: A deep learning model for source code vulnerability detection by integrating token-level and function-level features

Erzhou Zhu *, Xiangshan Qu, Xiaohan Liu, Xuejian Li

School of Computer Science and Technology, Anhui University, Hefei 230601, China

ARTICLE INFO

Keywords:
Vulnerability detection
Feature extraction
Code attention
Deep learning

ABSTRACT

Deep learning is widely used in vulnerability detection due to its high accuracy. However, existing models often fail to capture both token-level and function-level features. To address this limitation, a BERT-based Multi-Granularity Attention Network (BMGANet) is proposed. In the BMGANet model, Program Dependence Graphs (PDGs) are first constructed using the Joern tool, and Abstract Syntax Trees (ASTs) are extracted according to predefined vulnerability rules. Cross-user-defined-function program slicing and code normalization are then applied to enhance analysis efficiency. Processed code slices are fed into a BERT network to extract initial token-level and function-level features. To overcome BERT's limitation in modeling temporal dependencies, an LSTM network and a multi-head attention mechanism are sequentially employed to refine token-level features. The refined token-level features are then fused with function-level features for accurate vulnerability detection. Two pretraining tasks, namely the dynamic masked token prediction and the inter-code-line logical correlation prediction, are introduced to strengthen the model's ability to handle semantic gaps and weak logical connections. Experimental results on both synthetic and real-world datasets show that BMGANet outperforms state-of-the-art methods.

1. Introduction

In software development, security-related defects are collectively defined as vulnerabilities. Attackers can exploit these to launch malicious activities, including data tampering, information leakage, unauthorized control, and service destruction. The number of reported vulnerabilities has been rising steadily. By September 2025, the Common Vulnerabilities and Exposures (CVE) database had recorded 29,200 entries [1], more than four times the number in 2016. Most software attacks are launched by exploiting inherent design flaws [2]. Thus, vulnerabilities have become a core threat to digital system security.

Consequently, the accurate and efficient detection of software vulnerabilities remains a critical focus in program security. Currently, static and dynamic program analysis techniques, such as code auditing, pattern matching, and rule-based detection, are widely applied [3]. Nevertheless, these traditional methods heavily rely on domain expertise and complex rules. They often suffer from the state explosion problem, leading to limited performance, high false positive and negative rates, and difficulties in identifying complex vulnerability patterns.

With the rapid advancement of artificial intelligence and increasingly accessible computing resources, numerous machine learning methods have been proposed for software vulnerability detection, encompassing static, dynamic, and hybrid static-dynamic analysis scenarios [4]. Traditional machine learning approaches typically rely on feature engineering as the core means of vulnerability

* Corresponding author.
E-mail addresses: ezzhu@ahu.edu.cn (E. Zhu), xsqu001@163.com (X. Qu), u24301305@163.com (X. Liu), lxj@ahu.edu.cn (X. Li).

https://doi.org/10.1016/j.compeleceng.2026.110999
Received 7 October 2025; Received in revised form 14 January 2026; Accepted 26 January 2026
Available online 30 January 2026
0045-7906/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

identification. However, the accuracy of such systems is highly dependent on prior knowledge of relevant features. Moreover, extracting comprehensive, multi-dimensional features to achieve high detection accuracy is time-consuming, and attackers can bypass established detection systems by exploiting the limitations of manually designed features [5].

Deep learning, a pivotal branch of machine learning, has been extensively studied for data classification due to its powerful autonomous learning capabilities and independence from expert knowledge [6]. Nevertheless, two key challenges have long impeded its application in vulnerability detection: the complexity of vulnerability samples and the demand for large-scale datasets [7]. Recently, the proliferation of open-source communities has generated vast amounts of code-centric data. Consequently, leveraging deep learning to automatically obtain semantically rich code representations has emerged as a vital approach. Typically, these frameworks convert source code into vectors that capture vulnerability characteristics.

Current research on deep learning-based vulnerability detection primarily follows two technical paths: text sequence-based and graph structure-based methods [8]. Text sequence-based models apply Natural Language Processing (NLP) techniques to process source code. They input code tokens (e.g., keywords, identifiers) sequentially to learn syntactic and semantic features. These models require minimal preprocessing and effectively detect linear vulnerabilities. However, a critical limitation is that significant syntax and semantic information is lost when token sequences are converted into vectors. Since this information is crucial for detection performance, its absence frequently degrades practical effectiveness. Studies indicate that incorporating semantic information from data and control dependence can reduce the false negative rate by an average of 30.4 % [9].

To preserve more syntactic and semantic details, many models convert source code into graph structures. For instance, some methods integrate code features into a Code Property Graph (CPG) [10] and employ Graph Neural Networks (GNNs) to capture complex hierarchical information [11]. However, GNNs have inherent limitations in handling long-distance dependencies between non-adjacent nodes [12]. Although multi-layer GNNs can learn global information, they may cause over-smoothing, where nodes with different labels produce similar embeddings, thereby impairing performance [13]. Additionally, existing GNN models often over-rely on global features while neglecting local information and the importance of individual nodes.

To address the aforementioned limitations, this study proposes BMGANet (BERT-based Multi-Granularity Attention Network), a deep learning model for source code vulnerability detection that integrates token-level and function-level features. BMGANet shares methodological similarities with StagedVulBERT [14], a recently proposed representative model: (1) both are built on the BERT (Bidirectional Encoder Representations from Transformers) architecture; and (2) both adopt a coarse-to-fine strategy to extract and fuse multi-granularity features, enabling them to capture information across different levels of the target code. However, StagedVulBERT has limitations in capturing comprehensive global information of the target code. Specifically, it directly extracts statement-level features, which have a relatively narrow scope. To mitigate this constraint, the model utilizes a segmented Transformer encoder to process ultra-long token sequences.

By contrast, our proposed BMGANet implements the coarse-to-fine strategy in a distinct manner, primarily reflected in two key aspects: (1) To acquire a broader range of holistic information from the target code, we incorporate graph structures and code slicing into the preprocessing phase. Specifically, BMGANet first uses the Joern analysis tool to convert source code into a Program Dependence Graph (PDG), then extracts Abstract Syntax Trees (ASTs) that align with vulnerability rules. On this foundation, we perform cross-user-defined-function program slicing and code normalization. These steps can expand the scope of code analysis while preserving critical syntactic and semantic information. (2) Based on the preprocessed text sequences, we leverage the BERT model to extract token-level and function-level features. Through sequential optimization of token-level features and dual-level feature fusion, BMGANet generates feature vectors that accurately characterize vulnerabilities. The main contributions of this study are summarized as follows:

(1). *Proposes the BMGANet model*: BMGANet is a novel multi-granularity model that is specifically tailored for code slices to bridge pre-trained models with fine-grained analysis. Initialized with CodeBERT weights, the model inherits strong code understanding but overcomes semantic gaps in slice processing through a specialized pre-training strategy. To prevent overfitting to project-specific styles, we perform code normalization to force the model to learn generalizable vulnerability patterns rather than syntactic artifacts, thereby improving cross-project generalization.
(2). *Enhances model performance through multi-dimensional optimization*: BMGANet performs cross-user-defined-function program slicing and normalization to balance analysis efficiency and information retention. For feature fusion, a concatenation strategy is adopted to fully preserve global and local code features in the final feature vector. In the pretraining phase, two specialized tasks, namely the dynamic masked token prediction and the inter-code-line logical correlation prediction, are designed to strengthen the model's robustness against semantic gaps within slices and weak logical correlations between slices.
(3). *Verifies comprehensive model performance*: We implement BMGANet and conduct detailed performance evaluations. On both synthetic and real-world datasets, BMGANet outperforms mainstream comparative models in key metrics including accuracy, precision, recall, and F1-Score. For instance, its average F1-Score is 3 %–8 % higher in detecting vulnerabilities associated with four syntax rules; ablation experiments confirm that multi-granularity feature fusion significantly boosts performance (e.g., the F1-Score for CWE-119 increases from 86.29 % to 91.20 %); and the average accuracy for 10 typical Common Weakness Enumeration (CWE) vulnerabilities reaches 94.48 %, verifying the model's effectiveness and generalization ability.

The remainder of this paper is structured as follows: Section 2 reviews related work; Section 3 elaborates on the design and implementation of the BMGANet model; Section 4 presents experimental results and analysis; and Section 5 concludes the study and outlines future research directions.

2. Related work

The accurate and efficient discovery and detection of code vulnerabilities have long been a core research focus in information security. With the advancement of artificial intelligence and the growing accessibility of computational resources, machine learning methods are now widely adopted for software vulnerability detection.

Traditional machine learning-based vulnerability detection methods extract vulnerability-related features from programs either manually or automatically. They then identify vulnerabilities by combining techniques such as machine learning classification or rule matching. Their basic workflow typically includes four stages: data collection and preprocessing, feature extraction (encompassing syntactic, semantic, statistical, and dynamic features), feature selection and optimization, and model training and detection [15]. Compared with conventional methods like manual code review and rule-based matching, machine learning approaches offer distinct advantages. They enable high automation, eliminating the need for extensive manual feature definition or code inspection. They also adapt well to large-scale detection scenarios and support fast, batch processing after model training [16]. However, the performance of traditional machine learning methods has long been constrained by factors such as dataset quality, classification model design, vectorization techniques, and function/variable name replacement [17]. For example, the SmartMixModel [18] computes contextual information from the AST of source code and uses Random Forest (RF) and Support Vector Machine (SVM) as base classifiers to detect known vulnerabilities. Yet, deficiencies in its feature extraction process limit both its detection accuracy and the range of recognizable vulnerability types, leaving room for improvement and expansion.

Owing to their strong autonomous learning capabilities and ability to extract features without relying on expert knowledge, numerous deep learning-based methods have been proposed in recent years to enhance program vulnerability detection performance [19]. From the perspective of source code representation, current deep learning-based vulnerability detection models mainly follow two approaches: text-based representation and graph-structured representation [20].

2.1. Text-based vulnerability detection models

Text-based models treat source code as sequences of code tokens and apply NLP techniques for modeling. For instance, they use word embedding methods like Word2Vec and GloVe to convert code words or symbols into vector representations. Alternatively, they transform code snippets into high-dimensional vectors (as Code2Vec does) to capture semantic similarity between different code segments.

SySeVR [9] is the first framework that systematically applies deep learning to vulnerability detection in C/*C*++ source code. It first extracts Syntax-based Vulnerability Candidates (SyVCs) containing vulnerability-related syntax features. It then integrates data dependencies and control dependencies to generate Semantics-based Vulnerability Candidates (SeVCs) with semantic information. Finally, the vectorized SeVCs are fed into a deep learning model for vulnerability detection. VulDeeLocator [21] is a deep learning-based fine-grained vulnerability detector for C/*C*++ source code, focusing on high detection and localization accuracy. It extracts simplified Syntax-based Vulnerability Candidates (sSyVCs) from source code via four syntax types. It converts code to LLVM intermediate language to generate intermediate Semantics-based Vulnerability Candidates (iSeVCs), labeled with vulnerable line numbers or 0. After normalization, networks like BRNN-vdl are used for training and detection. While it outperforms other models on synthetic datasets in localization accuracy, its effectiveness lacks verification in real-world scenarios.

VulDeePecker [22] addresses the reliance of existing solutions on human experts for feature definition and their high false-negative rates. Its core innovation lies in representing programs with code gadgets (semantically related non-consecutive code lines). These gadgets are converted into vectors, forming a deep learning-friendly representation that adheres to the principles of applying deep learning to vulnerability detection. μVulDeePecker [23] optimizes VulDeePecker by introducing the concept of "code attention". This enables the model to capture information that helps pinpoint specific vulnerability types. When tested on the Software Assurance Reference Dataset (SARD) [24] and the National Vulnerability Database (NVD) [25], the aforementioned models demonstrate effectiveness in vulnerability detection. However, their performance in real-world scenarios remains unvalidated.

To capture both intra-statement vulnerability features and inter-statement long-term dependencies, Tao et al. [26] first generated bimodal slices through cross-slicing. They then developed a purification method that shortens source code slices to reduce interference from irrelevant statements and replaces obscure information in assembly code with clearer content. These steps ultimately help the model better capture vulnerability features.

2.2. BERT-based vulnerability detection models

Since source code is a sequence of symbols with strict order and state transitions, and these characteristics align well with BERT's processing mechanisms. As a result, BERT has been widely applied in code vulnerability detection research.

VulBERTa [27] is a deep learning approach designed for source code vulnerability detection. It pretrains a RoBERTa model to acquire in-depth knowledge of code syntax and semantics. This pretrained model is then used to train classifiers specifically tailored for vulnerability detection tasks. StagedVulBERT [14] is a multi-granular vulnerability detection framework that leverages a pretrained code language model and adopts a coarse-to-fine strategy. Built on BERT, its hierarchical, layered, and semantic encoding enables simultaneous capture of semantics at both the token and statement levels. Thapa et al. [28] proposed a vulnerability detection method using large transformer-based language models. This approach is grounded in three key foundations: the excellent performance of such models in natural language processing, the transferability of their acquired knowledge across related domains, and the inherent similarity between natural languages and high-level programming languages (e.g., C/*C*++). Notably, when CodeBERT is adopted as

the underlying transformer-based model, the method achieves optimal performance. The BERT module generates robust contextual embeddings for each code token. However, relying solely on its token-level output features hinders the explicit modeling of temporal dependencies and dynamic changes within source code.

2.3. Graph-based vulnerability detection models

Many deep learning vulnerability detection models convert source code into graph structures to integrate more semantic and syntactic information. These graph-based models construct structures such as AST, Control Flow Graphs (CFG), Data Flow Graphs (DFG), and PDG. They then use GNNs to extract complex features of nodes and their dependency relationships [12].

DeepWukong [29] obtains PDG subgraphs via a slicing algorithm. It enhances detection performance by incorporating structural information from eXtended Flow Graphs (XFGs). VDoTR [30] integrates AST, CFG, DFG, and Node Contextual Structures (NCS). It represents code graphs in a comprehensive tensor form, enabling the detection of multiple types of code vulnerabilities. To capture comprehensive vulnerability semantics, GraphFVD [31] extracts property graph-based slices from the CPG. It then uses a hierarchical attention graph convolutional network to learn graph embeddings. FUNDED [32] leverages GNNs to capture and reason about program control, data, and call dependencies. It also incorporates probabilistic learning and statistical evaluation to automatically collect high-quality training samples from open-source projects, thereby enhancing detection accuracy. VulDetector [33] is a static analysis tool for C/C++ vulnerability detection based on function-level graph comparison. It slices the function's CFG by locating vulnerability-sensitive keywords, generates weighted feature graphs containing both syntactic and semantic features, and detects vulnerabilities by comparing the similarity of these graphs.

To address the limitation that traditional graph-based methods predominantly rely on homogeneous graphs, Huang et al. proposed HeVulD [34]. This method effectively distinguishes between different types of code statements and dependencies, while also overcoming the shortcomings of traditional rule-based methods and certain deep learning approaches. SVulDetector [35] is a code similarity-based vulnerability detection method. It specifically targets the interference of irrelevant statements in code with identical vulnerability patterns and the challenge of detecting minimal differences between vulnerable code and its fixed version. Through Slice Composite Graphs (SCGs), tree-based attention mechanisms, and weighted graph embedding mechanisms, it precisely focuses on core syntactic and semantic information related to vulnerabilities, thereby enhancing the detection accuracy of vulnerabilities caused by code reuse.

To effectively identify vulnerabilities and locate vulnerable statements, FVD-DPM [36] formulates vulnerability detection as a diffusion-based graph-structured prediction task. It first extracts graph-level program slices from the Code Joint Graph (CJG) to generate fine-grained code representations. It then uses a conditional diffusion probabilistic model to model the node label distribution

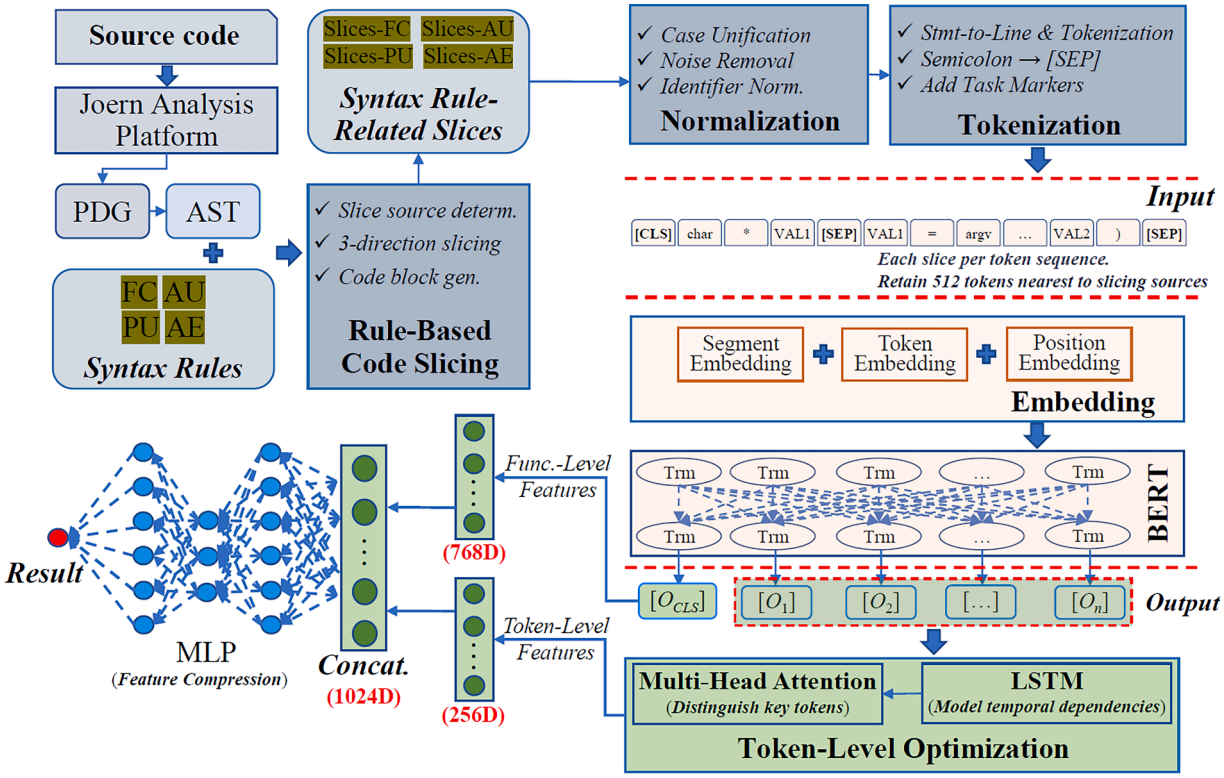

Fig. 1. BMGANet: A multi-granularity vulnerability detection model architecture tailored for code slices.

within these slices, achieving both slice-level vulnerability identification and statement-level vulnerability localization. To capture complete and accurate vulnerability semantics, SnapVuln [37] first extracts different types of vulnerability semantics using vulnerability-specific inter-procedural slicing algorithms. It then leverages a Gated Graph Neural Network (GGNN) with an attention mechanism to learn these semantics, ultimately enabling effective vulnerability detection.

3. The proposed BMGANet model

Based on the architecture depicted in Fig. 1, the operating process of BMGANet is detailed as follows:

(1). The code slicing and normalization module extracts code fragments containing key semantic information from the source code and standardizes the code within these fragments. This step ensures efficient input for subsequent code feature extraction while preserving critical semantic details.
(2). The feature extraction module leverages BERT's dual capability to process sliced code both as an integrated unit and in segmented parts, enabling the simultaneous extraction of function-level and token-level code features.
(3). The token-level feature optimization module addresses BERT's inherent limitation in sequence modeling by using LSTM to capture long-distance dependency information in the source code. To further enhance the model's ability to focus on key code regions, a multi-head attention mechanism is introduced to weight and optimize the token-level features output by LSTM. This allows the model to prioritize information regions relevant to vulnerabilities, reduce interference from redundant features, and improve both detection accuracy and efficiency.
(4). The feature fusion and classification module concatenates function-level features with optimized token-level features. The fused features are then fed into a classifier for training and prediction, enabling the accurate detection of potential vulnerabilities in the source code.

3.1. Code slicing and normalization

This study sequentially performs code slicing and normalization on the source code, aiming to provide structurally standardized, semantically rich, and interference-free input data for subsequent multi-granularity feature extraction.

3.1.1. Code slicing

To enable finer and more flexible vulnerability detection, code slicing is performed prior to source code tokenization.

(1). Vulnerability syntax rules

This study leverages C/C++ vulnerability rules from the commercial tool Checkmarx to analyze the syntactic characteristics of vulnerabilities. Since these rules cover most software vulnerabilities, vulnerability analysis is based on the following four key syntactic characteristics:

(a). *Library/API Function Call (FC)*: Focuses on whether the program contains calls to standard library functions or third-party APIs with known security risks, which may lead to buffer overflows and format string vulnerabilities.
(b). *Array Usage (AU)*: Concerns whether boundary checks are implemented for array indices and whether out-of-bounds access occurs after dynamic memory allocation. Failure to enforce these can cause memory corruption and information leakage.
(c). *Pointer Usage (PU)*: Refers to whether the program involves dereferencing uninitialized, dangling, or null pointers, which can trigger null pointer dereferencing and use-after-free vulnerabilities.
(d). *Arithmetic Expression (AE)*: Focuses on whether the program contains arithmetic operations involving integer overflows, sign errors, or division by zero, which may result in logic errors and abnormal memory allocation.

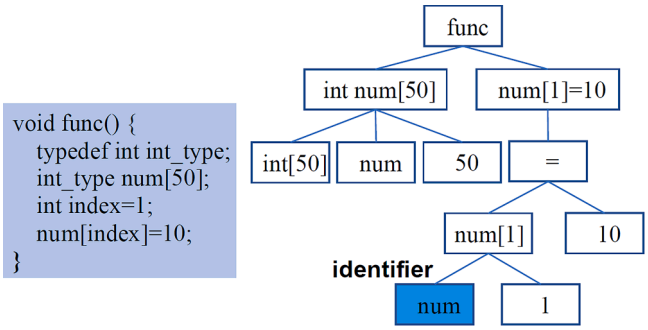

Fig. 2. Example of AU syntax characteristic in code snippets.

In implementation, one or more code slices are extracted from a function based on the number of statements matching the predefined vulnerability syntax rules. This study employs the Joern platform to generate ASTs for the functions within the target program. By analyzing these ASTs to identify syntax elements aligned with the four aforementioned rules, this step guides subsequent program slicing and normalization.

For example, in the code snippet on the left of Fig. 2, the variable "*num*" is declared in the statement "*int num[50];*" (containing array delimiters "*[*" and "*]*"), so it is identified as matching the AU syntax characteristic.

(2). Code slicing and code block generation

Since most vulnerabilities are closely tied to the four aforementioned syntax rules, code lines related to these rules are designated as primary slicing sources. To enable cross-function code slicing, we utilize the Joern tool to generate PDGs and ASTs. This approach ensures that relevant code fragments associated with cross-function vulnerabilities are encompassed within a single slice. Specifically, PDG token nodes corresponding to FC/AU/PU/AE AST elements are treated as slicing sources. Additionally, code differences between vulnerable and patched versions are incorporated, as they directly reflect vulnerability locations and characteristics.

In the PDG, both data dependency and control dependency information are encoded in the edges connecting different nodes. The type of slicing performed (forward, backward, or bidirectional) is determined by the characteristics of the slicing source:

(a). FC/AE-related sources: Backward slicing traces statements influencing the source (vulnerable code precedes risky operations like API calls).
(b). Patched code/PU-related sources: Forward slicing tracks statements affected by the source (impact extends post-operation).
(c). Composite/ambiguous sources: Bidirectional slicing retains maximum syntactic and semantic context to avoid losing critical vulnerability information.

Unlike methods that directly tokenize raw slices, this study processes slices into informative code blocks to ensure vulnerability logic integrity and reduce noise. Duplicate and overlapping slices are first removed and merged. Given that useful information positions vary across slices from different directions, code blocks are assembled as follows:

(a). Collect all slices from the same source;
(b). Remove duplicates to eliminate redundancy;
(c). Assemble by slicing direction: Single-direction slices (before/after the source) are integrated; bidirectional slices are aligned at the source, with duplicate statements removed to form a unified block.

Fig. 3 illustrates a code slicing example for FC-related code potentially causing a buffer overflow. On the left is the original code snippet: the risky *strcpy()* function (matching the FC syntax rule) is invoked with two parameters, namely the "*buf*" (declared as "*char buf[15]*" in line 4 of the user-defined *test()* function) and the "*str*" (originating from *argv[1]* in *main()* and passed as an input parameter to *test()*).

After applying backward slicing targeting the *strcpy()* call and removing duplicate statements, the right side of Fig. 3 shows the final sliced result. It retains only the core dependency chain critical to the buffer overflow risk: the definition of "*buf*" (*test()*, line 4), the assignment of "*str*" (*main()*, line 15), and the *strcpy()* operation (*test()*, line 8), effectively filtering out irrelevant code (e.g., the "*if(!buf)*" check in *test()*, line 6) that would introduce noise.

This example confirms that backward slicing captures statements influencing the *strcpy()* operation and highlights the model's cross-function capability, namely slices can include code from multiple user-defined functions (e.g., *main()* and *test()*), which is critical

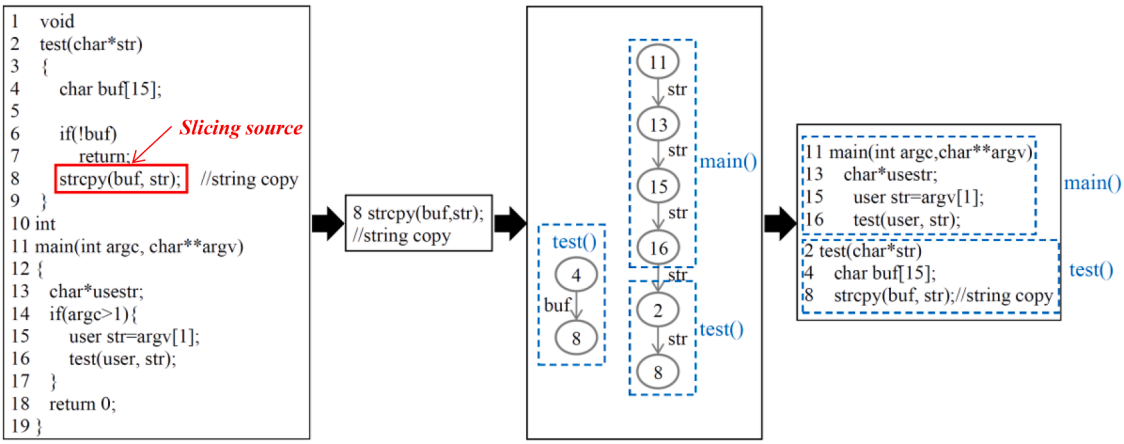

Fig. 3. Example of cross-function code slicing for FC-related buffer overflow vulnerability.

for detecting inter-function dependency vulnerabilities. Note that this example focuses on FC-related slicing. In practice, slices corresponding to AE, AU, PU, and other vulnerability syntax rules are also extracted from the same source file.

In practice, a single function may contain multiple statements that match different vulnerability syntax rules (e.g., both a risky API call and an arithmetic operation). In such cases, our preprocessing phase generates multiple independent slices for the same function, each centered on a specific slicing source.

During training and inference, BMGANet evaluates these slices individually. We adopt a "*Positive-in-Any*" strategy for function-level vulnerability prediction: if any slice derived from a function is classified as vulnerable by the classifier, the entire function is flagged as containing a vulnerability. This strategy ensures the model does not miss critical vulnerability patterns, even when a single function involves multiple potentially conflicting syntactic contexts.

3.1.2. Sliced code normalization

After generating code slices, this study performs normalization and standardization on the sliced code. By eliminating semantic-irrelevant noise, the process enhances data quality and improves the model's generalization ability. The normalization steps are detailed as follows:

(1). Case unification. Standardize the case of all characters in the sliced code. This ensures coding consistency and avoids interference from case differences during model training.
(2). Non-semantic character cleaning. Use regular expressions to automatically remove non-ASCII characters, comments, and formatting symbols. This guarantees the purity and consistency of the sliced content.
(3). Variable name normalization. In compliance with C11 and *C*++17 standards, keywords and main function parameters retain their original names. All user-defined variables are uniformly renamed to "*VAL{number}*" (e.g., "*VAL1*"), preventing semantic leakage or bias introduced by variable naming.
(4). Function name normalization. Except for the main function, all user-defined functions are uniformly renamed to *"FUN{number}"* (e.g., *"FUN1"*), ensuring consistency and controllability in function naming.

This study adopts a one-to-one mapping strategy to convert user-defined variable/function names into standardized representations. Notably, multiple variables mapped to the same standardized name are allowed to appear in different code slices. Fig. 4 illustrates the normalization process applied to the sliced code (derived from the right-hand side of Fig. 3).

3.2. Tokenization of sliced code blocks

This study adopts code slices as the input unit for BERT and extracts both token-level and function-level features from these slices, thereby enhancing the model's vulnerability detection performance.

The generated sliced code blocks are first subjected to tokenization. To ensure BERT receives a logically intact representation of vulnerability-related information, the fully preprocessed (standardized and tokenized) code slice is fed into BERT as a single token sequence. However, even though heuristic slicing based on syntactic rules and code normalization effectively reduces noise, the resulting slices may still exceed BERT's maximum input length of 512 tokens when handling code with extremely complex data or control flows. To accommodate this constraint, we implement a truncation strategy biased toward the slicing source. Guided by the principle that "*critical vulnerability information is typically localized near the slicing source*", we retain only the 512 tokens closest to and surrounding the slicing source. This approach preserves the most semantically relevant context for feature extraction, maintaining the

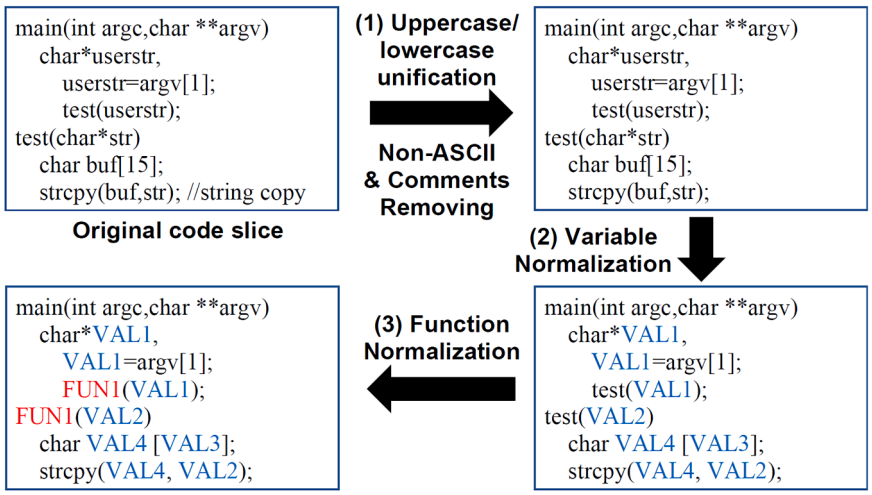

Fig. 4. Step-by-step normalization process for FC-related code slices.

integrity of the vulnerability pattern within the model's input limitations.

The specific tokenization process is implemented as follows:

(1). *Statement-level line conversion and tokenization*. To adapt to BERT's input format, all statements within a sliced code block are serialized into a single token sequence, with their original relative order preserved via position embeddings. Tokenization is then performed on these line-converted statements. For example, the sample code slice shown in the upper part of Fig. 5 can be sequentially split into tokens such as "*char*", "*", "*VAL1*", ",", "=", "*argv[1]*", ";", "*FUN1*", and "*VAL2*".
(2). *Semicolon replacement with "[SEP]" markers*. During tokenization, semicolons (which denote the end of statements in C/C++) are replaced with BERT's special "*[SEP]*" marker. This modification helps the model explicitly identify boundaries between consecutive statements, avoiding confusion caused by statement-level semantic discontinuity.
(3). *Addition of task-specific markers*. For each merged token sequence (comprising tokens from all statements in a code slice), two special markers are added to align with BERT's input format: (a) The "*[CLS]*" marker is prepended to the start of the sequence, serving as a placeholder for aggregated sequence-level features to support subsequent classification tasks; (b) The "*[SEP]*" marker is appended to the end of the sequence, explicitly marking the termination of the code slice and distinguishing it from other input sequences during batch processing.

To facilitate model processing, this study assigns unique indices to all tokenized symbols and constructs a token dictionary. Through this dictionary, any code slice can be converted into a sequence of token indices. Before feeding data into BERT to extract token-level and function-level features, the tokenized code slice input must first be converted into embedding vectors via three distinct embedding layers:

The tokenization result for each line of the finalized sliced code block is visualized in the top row of the lower section of Fig. 5. To facilitate model processing, all tokenized symbols are assigned unique indices to construct a token dictionary, enabling the conversion of any code slice into a sequence of token indices. Before feeding data into BERT for feature extraction, as shown in the low part of Fig. 5, the tokenized code slice must be converted into embedding vectors via three distinct layers:

(1). *Token embedding*. Converts each token in the sliced code into a dense embedding vector (e.g., "$E_{[CLS]}$" for the "*[CLS]*" marker, "E_{char}" for "*char*", "E_{VAL1}" for "*VAL1*").
(2). *Segment embedding*. Provides auxiliary metadata to distinguish which original statement each token belongs to (e.g., "E_A" for the first statement, "E_B" for the second), preserving statement-level structure even in merged sequences.
(3). *Position embedding*. Assigns unique vectors encoding each token's absolute position (e.g., "E_0", "E_1") using sine/cosine functions, addressing the self-attention mechanism's inability to prioritize sequence order.

As shown in Fig. 5, the final input to BERT is the sum of these three embeddings:

$$E_{Input} = E_{Token} + E_{Segment} + E_{Position} \tag{1}$$

here, $\boldsymbol{E}_{Token}$ captures semantic information of syntactic units; $\boldsymbol{E}_{Segment}$ (optional for single-sequence tasks) enhances structural clarity; $\boldsymbol{E}_{Position}$ ensures the model accounts for token order.

3.3. Function-level feature extraction

The BERT model takes vectorized token encodings as input. Function-level features are designed to capture the overall semantics of the code. Internally, BERT performs in-depth contextual modeling for each input token through a multi-layer bidirectional self-

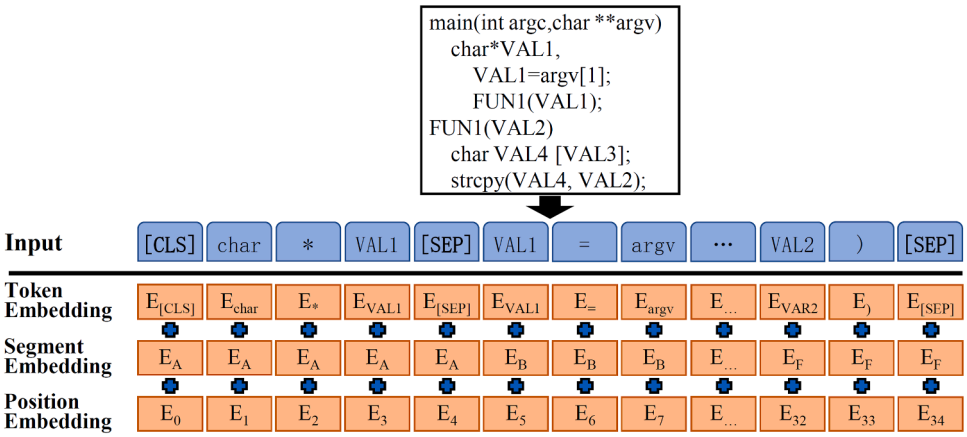

Fig. 5. Embedding layer processing for tokenized code slices (final BERT input representation).

attention mechanism. This allows the output feature of each token to not only retain its intrinsic information but also integrate contextual information from all other tokens within the same code slice. In turn, this achieves a comprehensive understanding of code semantics that supports both macro (function-level) and micro (token-level) feature extraction.

In practice, the design mechanism of BERT's "*[CLS]*" token is particularly well-suited to the demands of vulnerability detection, as demonstrated in the following aspects:

(1). *Inherent alignment with sequence-level classification.* The "*[CLS]*" token is explicitly designed for sequence-level classification tasks. During BERT's pre-training, the Next Sentence Prediction (NSP) task trains the "*[CLS]*" token to determine the coherence between two sentence sequences. This process inherently teaches the "*[CLS]*" token to synthesize the semantic information of the entire sequence and judge overall logical relationships. Consequently, this pre-training objective is highly consistent with our task of determining whether "an entire function constitutes a vulnerability".
(2). *Dynamic context-aware weighting via attention.* Within the Transformer's Self-Attention mechanism, the "*[CLS]*" token interacts with all other tokens and dynamically focuses on critical positions through attention weights. This mechanism acts as a "soft, context-aware aggregation" of key information, rather than a rigid averaging or truncation. As a result, the "*[CLS]*" token can better retain task-relevant critical information in classification tasks.

In practice, alternative strategies such as Mean Pooling (averaging all token vectors) and Max Pooling (extracting the most prominent features) are suboptimal for vulnerability detection. Mean Pooling dilutes the significance of "high-risk code fragments" by treating all tokens equally, while Max Pooling often severs the logical flow of code by retaining only the strongest signal. These limitations make them unsuitable for tasks requiring an understanding of the "complete execution path".

Based on this analysis, we directly utilize the "*[CLS]*" token as the function-level feature, denoted as $\boldsymbol{F}_{Func}$.

3.4. Token-level feature extraction and optimization

The context-aware embedding vectors $\boldsymbol{O}_i$ ($i = 1, 2, \ldots, n$) output by BERT for each token are directly defined as token-level features, denoted as $\boldsymbol{O}_{Token}$=($\boldsymbol{O}_1$, $\boldsymbol{O}_2$, …, $\boldsymbol{O}_n$). These features meticulously describe the grammatical functions and semantic roles of each syntactic unit within the global code context.

While BERT generates powerful contextual embeddings, it struggles to explicitly model temporal dependencies and dynamic changes in code slices. Source code is a sequence of symbols with strict order and state transitions. Meanwhile, semantics like variable definitions, assignments, and control flow transfers are closely tied to position. Temporal relationships (e.g., variable lifecycles, loop execution paths) are critical for detecting complex vulnerabilities, making BERT's standalone token-level features insufficient.

To address this, we integrate an LSTM network and a multi-head self-attention mechanism to optimize token-level features, enhancing the model's ability to capture complex semantics and key dependencies.

3.4.1. LSTM for temporal dependency modeling

The token-level features $\boldsymbol{O}^{Token} = (\boldsymbol{O}_1, \boldsymbol{O}_2, \ldots, \boldsymbol{O}_n)$ are fed into the LSTM, which generates output vectors for all time steps. The LSTM's gating mechanism and state updates are defined as follows:

$$f_t = \sigma\left(W_f \cdot \left[h_{t-1}, O_t^{Token}\right] + b_f\right) \tag{2}$$

$$i_t = \sigma\left(W_i \cdot \left[h_{t-1}, O_t^{Token}\right] + b_i\right) \tag{3}$$

$$\widetilde{C}_t = \tanh\left(W_C \cdot \left[h_{t-1}, O_t^{Token}\right] + b_C\right) \tag{4}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t, \tag{5}$$

$$O_t^{\text{Gate}} = \sigma\left(W_o \cdot \left[h_{t-1}, O_t^{Token}\right] + b_o\right) \tag{6}$$

$$h_t = O_t^{\text{Gate}} \odot \tanh(C_t), \tag{7}$$

where $\boldsymbol{O}_t^{Token}$ is the LSTM input at time step t; $\boldsymbol{h}_{t\text{-}1}$ is the previous hidden state; $\sigma()$ and $tanh()$ are activation functions; "$\odot$" denotes element-wise multiplication; and $\boldsymbol{W}_f$, $\boldsymbol{W}_i$, $\boldsymbol{W}_C$, and $\boldsymbol{W}_o$ and $\boldsymbol{b}_f$, $\boldsymbol{b}_i$, $\boldsymbol{b}_C$, $\boldsymbol{b}_o$ are learnable parameters.

After processing the entire sequence, the LSTM's output hidden state sequence and global representation are:

$$O^{\text{LSTM}} = [h_1, h_2, \ldots, h_D] \tag{8}$$

$$H_D = h_D \tag{9}$$

where $\boldsymbol{h}_1$, $\boldsymbol{h}_2$, …, $\boldsymbol{h}_D$ are hidden states at each time step t = 1, 2, …, D (with D as the final time step). These hidden states are concatenated in chronological order to form the sequence's feature representation. The LSTM integrates historical and current token information, capturing temporal dependencies like variable "assignment-use" relationships and conditional branching which are

critical for detecting complex vulnerabilities.

3.4.2. Multi-head self-attention for key feature focus

While LSTM models temporal dependencies, it treats all tokens equally and cannot explicitly prioritize key tokens (e.g., dangerous API calls, sensitive variables). We introduce a multi-head self-attention mechanism after the LSTM to weight and optimize key features, capturing richer inter-token dependencies across parallel subspaces.

Specifically, we establish relationships between the global representation $\boldsymbol{H}_D$ and $\boldsymbol{O}^{LS^{\text{TM}}}$:

(1). Linear transformations convert the LSTM output $\boldsymbol{O}_t^{LSTM}$ at time step t into ***Key*** and ***Value*** vectors:

$$Key_t = W_K \cdot O_t^{\text{LSTM}} \tag{10}$$

$$Value_t = W_V \cdot O_t^{\text{LSTM}} \tag{11}$$

(2). The global representation $\boldsymbol{H}_D$ is linearly transformed to generate a ***Query*** vector:

$$Query = W_Q \cdot H_D \tag{12}$$

(3). Correlation scores and attention weights are calculated:

$$e_t = \frac{Query \cdot Key_t^T}{\sqrt{d_k}} \tag{13}$$

$$a_t = \frac{\exp\,(e_t)}{\sum_{t=0}^{n} \exp\,(e_t)} \tag{14}$$

(4). Weighted summation of $\boldsymbol{Value}_t$ with a_t yields the self-attention-integrated code slice vector.

Since the LSTM already encodes positional information, no additional positional encoding is needed. This process leverages inter-time-step relationships in code slices, further improving detection performance and accuracy.

3.5. Feature fusion and classification

In deep learning-based source code vulnerability detection, effective fusion of multi-level and multi-granularity features is critical for enhancing model expressiveness and detection performance. BMGANet extracts two types of complementary features:

(1). Function-level features: High-level abstractions of global semantics, reflecting the overall structure, functionality, and contextual environment of code slices.
(2). Token-level features: Detailed depictions of syntactic and semantic roles of individual tokens, capturing microstructures like variable dependencies and control flow transitions.

Relying on either feature type alone struggles to balance global and local information. Thus, fusing them into discriminative comprehensive features is key to improving the detection performance. In this study, we adopt a concatenation strategy for feature fusion based on the following considerations: (1) It directly combines the original function-level (768D) and optimized token-level (256D) feature vectors without dimensionality reduction; (2) It fully preserves the integrity of global structural information (from function-level features) and local semantic details (from token-level features); (3) Its linear time complexity ensures low computational overhead, which is well-suited for our application scenario.

In specific implementation, let the BERT-output function-level feature be $\boldsymbol{F}_{Func} \in \mathbb{R}^{d_f}$ (where d_f is the feature dimension) and the LSTM+attention optimized token-level feature be $\boldsymbol{F}_{Token-LA} \in \mathbb{R}^{d_t}$ (where d_t is the feature dimension). The fused feature is expressed as:

$$F_{Fused} = [F_{Func}; F_{Token-LA}] \tag{15}$$

where "*;*" denotes concatenation, and $\boldsymbol{F}_{Fused} \in \mathbb{R}^{d_f+d_t}$. This method retains the original feature distribution and information, allowing the fully connected layer to learn complex inter-feature relationships.

In the final stage, the fused feature vector $\boldsymbol{F}_{Fused}$undergoes nonlinear transformation via a fully connected layer, followed by *Softmax* normalization to output class probabilities for vulnerability detection. The process is mathematically expressed as:

$$Z = \sigma\left(W_{fc} F_{Fused} + b_{fc}\right) \tag{16}$$

$$P = \text{Softmax}(W_{out} Z + b_{out}) \tag{17}$$

where $\boldsymbol{W}_{fc}$, $\boldsymbol{b}_{fc}$, $\boldsymbol{W}_{out}$, and $\boldsymbol{b}_{out}$ are learnable parameters, and $\sigma()$ is the activation function. During training, cross-entropy loss optimizes model parameters. A lightweight fully connected layer compresses the initial 1024-dimensional concatenated features, mitigating sparsity and insufficient interaction issues from direct concatenation.

3.6. Pretraining of BMGANet for code slices

Unlike pretraining on entire source code files, this study uses code slices to pretrain BMGANet, enhancing its performance in code analysis tasks. However, code slices suffer from internal semantic gaps and weak inter-slice logical correlations compared to full-source code.

To reduce training complexity while preserving basic code semantic understanding, the BERT component of BMGANet is initialized with CodeBERT's pretrained weights. As a BERT variant pretrained on complete programs, CodeBERT excels at code understanding but faces semantic gaps and weak logical coherence when directly applied to sliced code. To address this, we implement a domain-adaptive pretraining strategy that retains CodeBERT's unsupervised nature while introducing slice-centric objectives: dynamic masked token prediction and inter-code-line logical correlation prediction.

The overall joint loss function is:

$$L(\theta, \theta_1, \theta_2) = L_1(\theta, \theta_1) + L_2(\theta, \theta_2) \tag{18}$$

where θ denotes BERT Encoder parameters, θ_1 is the output layer parameter for the Masked Language Model (Mask-LM) task, and θ_2 is the classifier parameter for the sentence prediction task.

The pretraining of BMGANet via masked token prediction and inter-code-line logical correlation prediction aims to enable the model to learn general syntactic structures, semantic dependencies, and vulnerability-related feature patterns in source code, rather than project-specific characteristics. During pretraining, a unified code normalization strategy is applied: variable names are standardized as "*VAL{number}*", function names as "*FUN{number}*", and non-semantic elements (e.g., comments, formatting symbols) are removed. By stripping project-specific naming conventions and coding styles, this approach encourages the model to focus on universal vulnerability syntax/semantic patterns, thereby avoiding overfitting to project-specific features.

3.6.1. Dynamic masked token prediction

The masked token prediction task is designed to address semantic gaps in code slices. The standard BERT model employs a fixed masking strategy for masked token prediction, meaning the same tokens are masked across multiple training epochs. This fixed approach leads to inaccurate embedding of code tokens. To address this issue, this study adopts a dynamic masking technique, i.e., masking tokens dynamically in each training epoch, to enhance the model's robustness to unseen tokens in code slices. The workflow is as follows:

(1). *Token selection and masking*. Randomly select code fragments/keywords (e.g., function names, parameters, operators) to replace with a mask symbol. User-defined function names are excluded (their variability has minimal impact on detection). Key masked targets are determined by vulnerability types and programming languages.
(2). *Input conversion*. Convert the masked code fragments into BERT-compatible inputs using the method detailed in Section 3.2, then prompt the model to predict the tokens represented by the mask symbols.
(3). *Mask replacement and generalization enhancement*. Replace masks with the highest-probability tokens from the model's output. Multiple prediction rounds are allowed for suboptimal results to enhance generalization.

In the dynamic masking task, let M represent the set of masked code tokens in the first loss term L_1 of Eq. (18). This task is treated as a multi-classification problem with a vocabulary size of $|V|$, and its loss function is expressed as:

$$L_1(\theta, \theta_1) = -\sum_{i}^{|M|} log\ p(m = m_i | \theta, \theta_1) \tag{19}$$

where $m \in M$ denotes the original code token masked by the special MASK symbol, and $m_i \in V$ represents the candidate token from the vocabulary; the expression "m=m_i" indicates that the predicted token m_i matches the original masked token m.

The essence of this loss function is multi-class cross-entropy loss for the masked set M. Unlike the unidirectional prediction of word2vec, BERT's bidirectional Encoder can leverage contextual information to extract more robust features.

3.6.2. Inter-code-line logical correlation prediction

The task of predicting inter-code-line logical correlation is a critical component of model pretraining. The core value of this task first lies in targeting the structural characteristics of code slices to strengthen the model's learning of inter-code-line (i.e., inter-slice) correlations, ensuring code vectors from the embedding layer encode inter-line logical relationships. It also provides effective supervision signals for updating the parameters of BERT's self-attention layer (inherently for capturing code token correlations) via this binary classification task (judging code line adjacency). Which can help the layer identify code element dependencies more accurately. The implementation steps are:

(1). Prepend the "*[CLS]*" symbol to each code slice sequence (as a placeholder) and use "*[SEP]*" to separate internal data.
(2). Generate training samples following a "50/50″ principle: half are adjacent code lines (labeled "true"), and the other half are non-adjacent code lines (labeled "false").
(3). The model optimizes performance by predicting code line logical relationships (adjacent/non-adjacent) and updating parameters via backpropagation.

Although this task focuses on judging the order of code segments, its underlying goal is to enhance the model's ability to perceive the overall structure of code slices. The loss function for this task is expressed as:

$$L_2(\theta, \theta_2) = -\sum_{i=1}^{|M|} logp(n = n_i|\theta, \theta_2) \tag{20}$$

where, $n_i \in [IsNext, NotNext]$ indicates whether the code lines are adjacent; θ denotes the parameters of BERT's Encoder, and θ_2 represents the parameters of the output layer for this classification task. By minimizing this cross-entropy loss, the model can effectively learn the rules governing inter-code-line logical correlations.

3.6.3. Joint training for mask prediction and relationship judgment

The two tasks (L_1 and L_2) use the same batch of samples (each training iteration conducts both tasks on the same code fragment). The joint learning loss function is:

$$L(\theta, \theta_1, \theta_2) = -\sum_{i=1}^{|M|} (logp(m = m_i|\theta, \theta_1) + logp(n = n_i|\theta, \theta_2)) \tag{21}$$

4. Experiments and results

This section presents the experimental evaluation of BMGANet, aiming to validate its capability in detecting diverse vulnerability types defined by syntax rules, as well as its generalization performance on real-world software.

4.1. Experimental configuration

All experiments were conducted on a workstation equipped with an Intel Xeon 8474C CPU (2.10–3.80 GHz), an NVIDIA RTX 4090D GPU (24 GB memory), 64 GB of DDR5 RAM, and a 2 TB SanDisk SSD. The operating system used was Ubuntu 20.04 LTS. The BMGANet model was implemented using the PyTorch 2.0 framework with Python 3.10, leveraging CUDA 12.1 for GPU acceleration.

The hyperparameters were configured as follows: (1) The Adam optimizer was employed with an initial learning rate of 1×10^{-5} and a weight decay of 1×10^{-4}; (2) The batch size was set to 32, and the model was trained for 25 epochs with an early stopping strategy to prevent overfitting; (3) The BERT component was initialized with CodeBERT pretrained weights to ensure foundational code understanding; (4) The LSTM network contained 256 hidden units, and the multi-head attention mechanism utilized 8 heads; (5) A weighted cross-entropy loss function was adopted to address class imbalance.

4.2. Datasets and evaluation metrics

To construct the synthetic dataset (*N&S*) used in our experiments, we extracted C/*C*++ vulnerability samples encompassing ten representative CWE types (see Table 1) from the National Vulnerability Database (NVD) [25] and the Software Assurance Reference Dataset (SARD) [24].

As discussed in Section 3.1, the Joern analysis tool parses the source code to generate PDGs. Based on four key vulnerability syntax rules (FC, AU, PU, and AE), backward, forward, or bidirectional slicing extracts code fragments containing critical semantic information. Specifically, to eliminate noise and standardize the input, a rigorous multi-step normalization process is implemented:

Table 1
Dataset Composition and Statistical Distribution of *N&S* (Number of slices generated for each CWE type via the proposed preprocessing).

CWE-ID	Vulnerability Type	Samples	Generated slices
CWE-020	Input Validation Error	1450	2033
CWE-074	Injection	288	374
CWE-119	Buffer Error	1952	5075
CWE-311	Missing Encryption of Sensitive Data	376	382
CWE-399	Resource Management Error	1480	2368
CWE-400	Uncontrolled Resource Consumption	1082	1948
CWE-404	Improper Resource Shutdown or Release	916	1053
CWE-476	Null Pointer Dereference	1512	1642
CWE-665	Improper Initialization	297	356
CWE-704	Incorrect Type Conversion	815	1019

(1). *Label De-biasing (Crucial for Label Bias).* The "*Bad/Good*" labeling convention in the SARD dataset and the "*Vulnerable*" labeling convention in the NVD dataset are explicitly addressed. For instance, as shown in Fig. 6, the SARD dataset marks vulnerable code slices as "*bad*" and non-vulnerable ones as "*good*". To prevent the model from simply learning these literal labels for classification (which would limit its ability to generalize based on code semantics), a refinement in the data cleaning process removes markers like "*bad*", "*good*", or "*Vulnerable*" from function names. This ensures the model focuses on inherent code features rather than artificial labeling cues.
(2). *Structural Normalization.* As discussed in Section 3.1.2, all characters are converted to lowercase, non-ASCII characters and comments are removed, and variable/parameter names are standardized as "*VAL*<*number*>", while function names are standardized as "*FUN*<*number*>".
(3). *Deduplication.* As discussed in Section 3.1.1, during the code block generation phase, duplicate slices are removed and overlapping slices from different directions are merged to ensure data uniqueness.

These processes specifically target vulnerable code logic while excluding irrelevant code, resulting in the *N&S* dataset comprising 10,559 samples. Since the input units are code slices (rather than whole files), and slices from the same file but different functions are treated as independent samples after normalization, there is no overlap of semantic code blocks between the training and testing sets. Following the extraction of ASTs and PDGs using the Joern tool, cross-user-defined-function program slicing and normalization are performed. Consequently, as listed in the 4th column of Table 1, this process generates 16,250 distinct code slices that are formatted and normalized for input into the BERT model.

For model training and evaluation, 90 % of these slices are randomly allocated to the training set, the remaining 10 % serve as the testing set, and an additional 10 % of the training set samples are designated as the validation set to monitor training progress and prevent overfitting.

To verify the generalization capability of the proposed BMGANet model, further experiments were conducted on two real-world datasets, *Devign* [12] and *REVEAL* [38]. The *Devign* dataset was collected from four open-source repositories: Linux, FFmpeg, QEMU, and Wireshark. To improve computational efficiency, the dataset creators excluded functions with more than 500 nodes from the initial collection (which included 330,492 non-vulnerable functions and 18,945 vulnerable functions), resulting in a final dataset of 419,437 samples. The *REVEAL* dataset, a real-world vulnerability dataset compiled by Saikat et al., was sourced from vulnerabilities reported by developers and users of the Chromium and Debian projects. It contains 2240 vulnerable functions and 20,494 non-vulnerable functions, whose distribution is more consistent with that of vulnerable code in real-world scenarios. To achieve a precise evaluation of the model's performance across various metrics, the samples from the Devign and REVEAL datasets were subjected to the same preprocessing pipeline applied to the *N&S* dataset.

In the experiments presented in this section, vulnerable code is treated as positive samples, while non-vulnerable code is treated as negative samples. We employed four primary evaluation metrics: Accuracy (*A*), Precision (*P*), Recall (*R*), and F1-Score (*F1*). For real-project datasets, the False Positive Rate (*FPR*) was also calculated.

4.3. Training overhead of BMGANet

Under the experimental configuration detailed in Section 4.1, this section analyzes the time consumption of individual modules within the BMGANet model during a single training pipeline. The total time is primarily composed of three phases: data preprocessing, model training (25 epochs), and model validation.

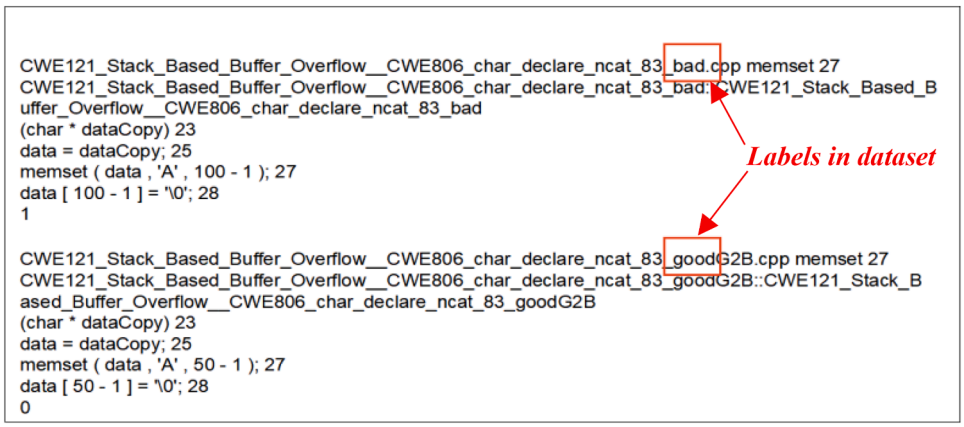

Fig. 6. Example of a sample in the SARD dataset.

(1). *Data Preprocessing.* The workflow in this phase includes: 1) Parsing C/*C*++ source code and constructing PDGs using Joern; 2) Performing code slicing (both forward and backward) to extract vulnerability-relevant code fragments; 3) Normalization and tokenization; 4) Outputting structured data for the PyTorch DataLoader.

Affected by code complexity, Gremlin query efficiency, and SSD I/O speed, the processing time for a single sample (including PDG construction and slicing) ranges from 2 to 4 s. Consequently, as listed in Table 2, processing the 10,559 samples (16,250 slices) in the *N&S* dataset consumed a total of 63 min.

(2). *Model Training.* The computational workload in this phase involves: 1) Utilizing the CodeBERT-based BERT encoder to generate semantic embeddings from code token sequences; 2) Employing an LSTM network (256 units) to model the sequential characteristics of code slices; 3) Applying a multi-head attention mechanism (8 heads) to fuse semantic relationships across different slices; 4) Adopting a weighted cross-entropy loss function to address class imbalance.

Approximately 90 % of the *N&S* dataset (9503 samples/16,479 generated slices) was used for training. With a batch size of 32, there were 297 steps per epoch, totaling 7425 steps over 25 epochs. Constrained by the computational capacity and memory bandwidth of the RTX 4090D, the average time per step (including forward pass, backward pass, and parameter update) was approximately 50–70 milliseconds. As a result, as listed in Table 2, the total time for the model training phase was 9.35 min.

(3). *Model Validation.* As indicated in Table 2, this phase consumed 2.26 min. This refers to the time required to evaluate the final model performance on the test set (constituting 10 % of the *N&S* dataset) after training is complete. The specific time consumption include: 1) Inference computation time: The time taken to feed each test sample into the trained BMGANet model, execute the forward pass, and output the prediction (vulnerable or non-vulnerable); 2) Data loading time: Encompassing reading preprocessed data from the disk, batching the data, and transferring it between CPU memory and GPU memory.

4.4. CWE-specific detection performance of BMGANet

We evaluated BMGANet on the 10 CWE vulnerability types listed in Table 1. As shown in Table 3, BMGANet achieves an accuracy exceeding 92 % across all vulnerability types. Specifically, it attained the highest accuracy of 96.83 % on CWE-476 and the lowest of 92.31 % on CWE-074. The average Precision, Recall, and F1-Score across all types are 92.15 %, 86.47 %, and 89.19 %, respectively, indicating a robust balance between false positives and false negatives.

BMGANet demonstrates superior performance in detecting structured vulnerabilities such as CWE-404, CWE-476, CWE-119, and CWE-665. This is attributed to the model's ability to capture distinct syntactic markers through multi-granularity feature extraction and the attention mechanism.

However, performance is comparatively lower for complex vulnerabilities requiring deep data flow or cross-module analysis, such as CWE-311 and CWE-704, where the F1-Score is 85.23 % and 86.18 %, respectively. This limitation stems from the constraints of slicing granularity and context windows, which may fail to capture long-range dependencies or implicit semantic relationships inherent in these vulnerability types.

4.5. Performance of BMGANet on four syntax rules

We further evaluated BMGANet's capability to identify vulnerabilities associated with the four syntax rule categories (FC, AE, AU, and PU) against five state-of-the-art models: VulDeePecker [22], SySeVR [9], mVulSniffer [39], FUNDED [32], and VDoTR [30]. In fact, validated by SySeVR, these four rules cover the vast majority of software vulnerabilities in this dataset. Specifically, FC covers 106 CWEs (e.g., strcpy-related buffer overflows); AU covers 87 CWEs (e.g., out-of-bounds array access); PU covers 103 CWEs (e.g., dangling pointer dereference); and AE covers 45 CWEs (e.g., integer overflows). The experimental results are summarized in Table 4. The results demonstrate that these four rule categories effectively capture vulnerabilities within the synthetic dataset (*N&S*).

It is worth noting that VulDeePecker, FUNDED, and VDoTR suffer from unavailability of complete source code and insufficiently detailed training parameter configurations. Consequently, only the methodologies described in the respective papers were reimplemented, and training parameters were re-tuned. In all cases, the experimental results in this study exhibit a certain degree of deviation from those reported in the original publications.

The input formats of the compared models differ, yet they share the same data foundation. Specifically, BMGANet, VulDeePecker, and SySeVR all take text sequences as input (with BMGANet incorporating PDG/AST information, while the others use shallow slices),

Table 2
Time consumption of BMGANet in the training phase.

Module	Time Consumption	Proportion
Data Preprocessing	63.43 min	84.53 %
Model Training	9.35 min	12.46 %
Model Validation	2.26 min	3.01 %
End-to-End Total	75.04 min	100 %

Table 3
Detection Performance of BMGANet on 10 Typical CWE Vulnerabilities (%).

CWE-ID	A	P	R	F1
CWE-020	95.82	93.20	87.76	90.41
CWE-074	92.31	90.55	86.98	88.73
CWE-119	95.15	95.52	87.75	91.20
CWE-311	92.61	88.45	82.24	85.23
CWE-399	94.33	93.65	89.91	91.74
CWE-400	93.92	91.17	86.36	88.70
CWE-404	96.24	94.81	86.20	90.31
CWE-476	96.83	93.12	88.02	90.50
CWE-665	94.51	91.74	86.25	88.91
CWE-704	93.12	89.31	83.26	86.18
Average	94.48	92.15	86.47	89.19

whereas VDoTR and FUNDED employ code graph structures based on Graph Neural Networks (GNNs). It is worth noting that due to the heterogeneous input representations and training methodologies, the tested models exhibit significant variance in training time and pre-training data scale. Specifically, the pre-training data (unlabeled code) serves to establish a general foundation of "code understanding" and does not contain task-specific knowledge regarding the vulnerability types present in our evaluation sets.

To ensure a fair comparison, the following experimental settings were established:

(1). All models are evaluated on the identical *N&S* test set (10 % of samples, as detailed in Section 4.2) under a standardized binary classification task, i.e., determining the presence or absence of vulnerabilities in the target code.
(2). All models, including pre-trained variants (BMGANet, VulBERTa), are pre-trained/fine-tuned on the exact same training samples (90 % of the *N&S* dataset). This ensures that the comparison focuses on the models' ability to learn from the same labeled data, regardless of their pre-training history.
(3). The experiment does not compare training time or pre-training data scale, but rather focuses on how effectively each model learns from the limited vulnerability data.

As shown in Table 4, BMGANet outperforms all comparative models across all metrics for all four syntax rules. This superiority is rooted in its architecture, which combines BERT's global semantic extraction, LSTM's temporal dependency modeling, and the multi-head attention mechanism's focus on critical syntax nodes. Specifically:

(1). *For the FC syntax rule*. BMGANet achieved an accuracy of 95.02 % and an F1-Score of 92.09 %. The model effectively identifies buffer overflows caused by API misuse by focusing on risky API nodes.
(2). *For the AE syntax rule*. BMGANet scored an F1 of 92.46 %, excelling at modeling the semantics of variables and operators to detect integer overflows.
(3). *For the AU and PU syntax rule*. The model captured latent dependency chains related to array bounds and pointer operations, achieving F1-Scores of 91.91 % and 92.19 %, respectively.

In contrast, models like VulDeePecker and mVulSniffer, which rely on RNNs or shallow features, struggle with global context. Graph-based models like VDoTR and FUNDED, while strong, sometimes sacrifice local detail for global structure. Although BMGANet consistently outperforms all comparative models across all metrics and under all four syntax rules, it still exhibits deficiencies in identifying cross-rule composite vulnerabilities, as well as those dependent on I/O information or requiring dynamic runtime contexts. This is primarily attributable to the limited coverage scope of the syntactic rules and the inherent constraints of static analysis.

To further evaluate BMGANet's coverage across the eight predominant vulnerability categories (acknowledging the existence of other categories but focusing on these due to their higher prevalence), we conducted tests using instances from the CWE Top 100 (2023-2025). Table 5 presents the detailed statistical data and testing results (Accuracy). As shown in the table, for vulnerability categories such as memory operations, pointer and memory management, and arithmetic and numerical processing, BMGANet is able to accurately detect the presence of vulnerabilities, benefiting from the comprehensive coverage and synergy of the four rule types.

Regarding code generation and execution vulnerabilities, although FC rules can identify dangerous function calls (e.g., *system, exec, eval*), the accuracy rate is only 79.7 %. This is because these vulnerabilities often involve dynamic code generation and execution processes, which are difficult for static analysis to fully capture. For file and path operation vulnerabilities, FC rules can only identify basic path concatenation operations. However, merits BMGANet's strong context-awareness at the logic level, enabling it to recognize complex path abuse scenarios such as symbolic link attacks, the detection accuracy reaches 86.5 %. As for input validation and sanitization vulnerabilities, although FC rules can cover related patterns to some extent, static methods struggle to determine whether inputs have been properly sanitized. Consequently, BMGANet fails to achieve high-accuracy detection for this category. For access control and permission vulnerabilities, the lack of corresponding rule coverage means BMGANet relies solely on its contextual understanding capability. As a result, it can only detect less than half of the samples. In the case of resource management vulnerabilities, the coverage provided by AE and FC rules leads to a relatively high overall accuracy. However, certain logic flaws (such as resource exhaustion due to uncontrolled loop execution) that are difficult to capture statically, which drag down the overall accuracy.

Table 4
Performance comparison of different models on four syntax rules (%).

Models	FC				AE				AU				PU			
	A	P	R	F1	A	P	R	F1	A	P	R	F1	A	P	R	F1
VulDeePecker	82.33	87.13	78.96	83.17	83.25	88.93	80.91	84.73	81.74	84.32	77.26	80.64	85.54	86.25	79.6	81.14
SySeVR	87.21	90.15	79.09	84.26	86.62	90.34	83.74	86.91	85.28	88.41	82.76	85.49	88.41	89.12	82.33	85.59
mVulSniffer	92.74	92.02	87.3	89.6	92.63	91.75	84.29	87.86	90.11	91.23	84.21	87.58	90.15	91.57	84.83	88.07
FUNDED	93.73	92.78	89.42	91.07	93.81	93.05	85.43	89.08	92.31	92.56	87.67	90.05	92.41	93.78	87.51	90.54
VDoTR	94.17	93.02	89.45	91.2	94.47	93.72	87.45	90.48	93.53	92.91	88.57	90.69	93.95	94.61	88.09	91.23
BMGANet	**95.02**	**94.15**	**90.11**	**92.09**	**96.64**	**95.87**	**89.28**	**92.46**	**95.22**	**93.91**	**90.11**	**91.97**	**95.93**	**95.12**	**89.43**	**92.19**

Table 5
BMGANet's vulnerability detection performance on 8 typical categories from CWE Top 100.

Vulnerability Category	Description	Quantity Ratio	CVE Examples	Involved Rules	Accuracy (100 %)
Memory Operations	Involves read/write operations on buffers, arrays, and strings	13 %	CVE-121, CVE-787	FC, AU	97.85
Pointers and Memory Management	Involves dynamic memory allocation, deallocation, and pointer operations	10 %	CVE-415, CVE-416	PU	98.26
Arithmetic and Numeric Processing	Involves integer and floating-point arithmetic and type conversions	6 %	CVE-190, CVE-682	AE	100
Code Generation and Execution	Involves dynamic code/command concatenation and execution	4 %	CVE-78, CVE-94	FC	79.7
File and Path Operations	Involves file path concatenation and access control	4 %	CVE-22, CVE-73	FC	86.5
Input Validation and Sanitization	Involves validation, filtering, and special character handling of user input	22 %	CVE-79, CVE-89	FC	67.2
Access Control and Permissions	Involves logic related to permission checks and identity authentication	16 %	CVE-284, CVE-862	None	48.6
Resource Management	Involves management of resources such as file handles, memory, and connections	5 %	CVE-404, CVE-770	AE, FC	82.5

In summary, BMGANet demonstrates strong detection capabilities within the scope of existing rule coverage. However, there remains room for improvement in handling vulnerability types that are highly dependent on dynamic behavior or complex contextual reasoning.

4.6. Ablation experiments

In this section, we conduct ablation studies to validate the contribution of each component of BMGANet, using samples from CWE-119 and CWE-399.

4.6.1. Impact of code slicing and normalization

This section presents experiments to validate the impact of the code slicing and normalization module on the overall performance of the BMGANet model. Two specific experimental setups were designed: (1) The BMGANet model with code slicing and code block generation removed, denoted as *BMGANet-NoSlice*; (2) The BMGANet model with sliced code normalization and dataset de-labeling removed, denoted as *BMGANet-NoNorm*.

Table 6 summarizes the test results of these two setups. For comparison, the performance metrics of the full BMGANet model (with all code slicing and normalization steps retained) are included in the last row of the table. As evident from the data in Table 6, removing either of the two preprocessing steps (code slicing/code block generation or normalization/de-labeling) significantly degrades the model's performance.

On one hand, three-directional code slicing and code block generation prune irrelevant code segments while preserving critical information necessary for vulnerability detection. This step is crucial for optimizing feature extraction efficiency and reducing interference from redundant data. When code slicing and code block generation are omitted (*BMGANet-NoSlice*), redundant and irrelevant content in the source code is incorporated into the feature extraction process. This leads to a series of cascading issues: (1) The model struggles to focus on vulnerability-related core logic; (2) False positive and false negative rates increase; (3) Key performance metrics, including accuracy, precision, recall, and F1-Score, demonstrate a clear decline.

On the other hand, the process of standardizing code formatting and eliminating non-critical syntactic variations, such as discrepancies in variable naming conventions or whitespace indentation, ensures the model can focus on the core syntactic logic pertinent to vulnerabilities, rather than being diverted by trivial formatting differences. Additionally, the dataset de-labeling procedure mitigates interference from annotation noise or redundant labels during model training, preventing the model from learning information irrelevant to vulnerability detection (e.g., artificial labeling suffixes like "*bad*" or "*good*" in dataset filenames). Omitting these two operations (*BMGANet-NoNorm*) harms performance in two main ways: (1) Unnormalized sliced code may mislead the model during feature extraction, making it harder for the model to learn generalizable vulnerability patterns; (2) Residual noise labels in the dataset introduce interference, increasing the model's misjudgment probability. This performance degradation is particularly pronounced

Table 6
Impact of code slicing and normalization on model performance (%).

Models	CWE-119				CWE-399			
	A	P	R	F1	A	P	R	F1
BMGANet-NoSlice	91.82	91.75	89.18	90.88	90.41	89.36	88.28	89.95
BMGANet-NoNorm	90.52	90.06	87.43	89.78	89.63	88.44	87.96	90.21
BMGANet	**95.15**	**95.52**	**87.75**	**91.20**	**94.34**	**92.78**	**89.91**	**91.32**

when detecting vulnerabilities sensitive to code logic details, such as CWE-119 and CWE-399.

4.6.2. Impact of feature extraction strategies

This section presents experiments to validate the impact of different feature types on the overall performance of the BMGANet model. Five specific experimental setups were designed:

(1). *BMGANet-Func*, a variant of BMGANet that exclusively uses function-level features (directly utilize the *"[CLS]"* token as the function-level feature) for vulnerability detection;
(2). *BMGANet-Func-MeanPool*, a variant of BMGANet that relies solely on function-level features derived from a Mean Pooling strategy. Specifically, rather than extracting the *"[CLS]"* embedding, this variant computes the arithmetic mean of the output vectors (embeddings) for all tokens in the code slice. This mean vector serves as the global representation of the function's semantics;
(3). *BMGANet-Token*, a variant that relies solely on token-level features for vulnerability detection;
(4). *BMGANet-Token-LSTM*, a variant that utilizes token-level features processed by an LSTM network for vulnerability detection;
(5). *BMGANet-Token-LSTM-Atten*, a variant that refines token-level features through a sequential integration of LSTM and multi-head self-attention mechanisms for vulnerability detection.

Table 7 lists the test results of the five groups of experiments on the CWE-119 and CWE-399 vulnerability datasets, respectively. For comparison, the performance metrics of the full BMGANet model are included in the last row of the table. The experimental results clearly demonstrate that using either function-level or token-level features in isolation yields inferior detection performance compared to fusing both feature types. Specifically, *BMGANet-Fucn* can capture the global structure and overall semantics of code slices, but it exhibits low sensitivity to micro-level information such as local code details and variable dependencies. This limitation directly leads to lower recall and F1-Score, making the model prone to missing complex or hidden vulnerabilities.

Experimental results also demonstrate that *BMGANet-Func-MeanPool* exhibits significantly inferior detection performance compared to *BMGANet-Func* based on *"[CLS]"* token embeddings. This performance gap stems from the fact that the Mean Pooling strategy, as a simplistic feature aggregation method, discards the global classification semantics inherent in the *"[CLS]"* token, rendering the model incapable of distinguishing critical vulnerability features from irrelevant code noise. Furthermore, this forced averaging operation dilutes the feature responses of high-risk code snippets, thereby substantially undermining the model's sensitivity and accuracy in detecting complex vulnerability patterns.

In contrast, *BMGANet-Token-LSTM* achieves superior performance because the LSTM compensates for the lack of inherent temporal modeling in BERT token-level features. Furthermore, building upon *BMGANet-Token-LSTM*, the *BMGANet-Token-LSTM-Atten* variant incorporates a multi-head self-attention mechanism. This integration strengthens the weights of vulnerability-relevant key tokens, thereby further improving detection performance.

4.6.3. Impact of feature fusion methods

This section presents experiments to validate the impact of different feature fusion strategies on the vulnerability detection performance of the BMGANet model. As defined in Eq. (15), let the function-level feature output by BERT be $\boldsymbol{F}_{Func} \in \mathbb{R}^{d_f}$ (where d_f denotes the dimension of the function-level feature vector), and the token-level feature optimized by the LSTM + attention mechanism be $\boldsymbol{F}_{Token-LA} \in \mathbb{R}^{d_t}$ (where d_t denotes the dimension of the optimized token-level feature vector). The BMGANet adopts concatenation as the default fusion method to balance efficacy and computational efficiency. To rigorously validate this choice, we benchmark five representative fusion strategies:

(1). *Summation*. Features are fused via element-wise addition (Eqation (22)):

$$F_{Sum} = \mathbf{F}_{Fucn} + \mathbf{F}_{Token-LA} \tag{22}$$

(2). *Tensor-Based Fusion*. Maps features to high-dimensional tensor space for high-order interactions (Eq. (23)):

$$F_{Tensor} = (F_{Func} W_1) \otimes (F_{Token-LA} W_2) \tag{23}$$

Table 7
Impact of different features on model performance (%).

Models	CWE-119				CWE-399			
	A	P	R	F1	A	P	R	F1
BMGANet-Fucn	93.31	89.43	83.37	86.29	91.67	88.79	83.28	85.95
BMGANet-Func-MeanPool	89.48	88.22	80.51	84.19	85.97	82.03	77.03	79.45
BMGANet-Token	94.78	92.42	86.38	89.30	92.73	91.93	88.36	90.11
BMGANet-Token-LSTM	94.81	93.02	86.39	89.58	93.04	91.95	88.71	90.30
BMGANet-Token-LSTM-Atten	94.87	93.02	86.69	89.78	93.04	91.99	88.84	90.60
BMGANet	**95.15**	**95.52**	**87.75**	**91.20**	**94.34**	**92.78**	**89.91**	**91.32**

where$\otimes$is the outer product, $\boldsymbol{W}_1 \in \mathbb{R}^{d_f \times r_f}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_t \times r_t}$ reduce dimensions to $r_f \ll d_f, r_t \ll d_t$.

(3). *Weighted fusion*. A linear combination with fixed weights (Eq. (24)):

$$F_{Weighted} = \alpha \cdot F_{Fucn} + \beta \cdot F_{Token-LA} \tag{24}$$

where $\alpha = \beta = 0.5$.

(4). *Gated Fusion*. Controls feature contribution ratios using gating (Eq. (25)):

$$F_{Gated} = \sigma\left(W_g \cdot [F_{Func}; F_{Token-LA}]\right) \odot \tanh(W_o \cdot [F_{Func}; F_{Token-LA}]) \tag{25}$$

where$\sigma$is the sigmoid function, and $\odot$ denotes element-wise multiplication.

(5). *Cross-Attention Fusion*. Dynamically highlights critical features via attention mechanisms (Eq. (26)):

$$F_{\text{Cross-atten}} = \alpha V \tag{26}$$

where $\alpha = softmax\left(QK^T / \sqrt{d_k}\right)$, $Q = F_{Func} W_Q$, $K = F_{Token-LA} W_K$, $V = F_{Token-LA} W_V$.

Experiments were conducted on samples from the *N&S* dataset corresponding to CWE-119 and CWE-399. The results are presented in Table 8, where "*Time cost*" represents the average detection time for samples in the *N&S* dataset. For comparative purposes, the performance metrics of the full BMGANet model (*BMGANet-Concatenation*) where function-level and token-level features are integrated via concatenation are included in the last row of the table.

The summation method simply performs element-wise addition of function-level and token-level features. This approach risks information loss due to feature overlap or mutual cancellation, which directly results in suboptimal recall and F1-Scores. In fact, the summation fusion in Eq. (22) is a special case of tensor-based fusion (specifically, first-order interaction). As shown by the test data in the table, the model's detection performance does not improve significantly when using the Tensor-Based Fusion method. Because the Tensor-Based Fusion method is better suited for handling the fusion of graph-structured features, while the function-level and token-level features involved in this study are both textual features, the performance improvement is limited. However, due to its high time complexity $O(d \cdot r + r^2)$ (where d is the dimension after concatenating token-level and function-level features, and r is the dimension after reduction), the average detection times for CWE-119 and CWE-399 are 122 *ms* and 187 *ms*, respectively.

The weighted fusion strategy with α and β both set to 0.5 balances the contribution of the two feature types but still compresses feature dimensions, limiting its ability to capture fine-grained differences between code patterns related to vulnerabilities. In fact, the weighted fusion in Eq. (24) can be viewed as a linear simplification of gated fusion (without nonlinear activation). After introducing the gating mechanism to control the contribution ratios, the model's detection performance improves to a certain extent compared to the weighted fusion strategy. The feature fusion scenario involved in this study is relatively simple, involving only the fusion of textual function-level (768D) and token-level (256D) features. Therefore, in this specific scenario, the performance improvement from more complex feature fusion techniques is relatively limited. However, due to the high time complexity $O(d^2)$ of Gated Fusion, the average detection times for CWE-119 and CWE-399 are 90 *ms* and 117 *ms*, respectively.

The Cross-Attention Fusion method highlights critical features through an attention mechanism and is suitable for capturing cross-granularity dependency relationships; thus, it is also applicable to the function-level and token-level textual features involved in this study. From the experimental data in the table, it can be seen that the performance of *BMGANet-Cross-atten* is close to that of the final

Table 8
Comparative experiments of feature fusion methods (%).

Models	CWE-119					CWE-399				
	A	P	R	F1	Time cost	A	P	R	F1	Time cost
BMGANet-Sum	94.31	93.28	86.65	89.84	35*ms*	93.94	93.28	87.26	90.17	56*ms*
BMGANet-Tensor	95.06	94.03	87.74	90.78	122*ms*	**94.59**	93.32	87.67	90.41	187*ms*
BMGANet-Weighted	94.73	94.38	87.29	90.70	43*ms*	93.66	93.53	88.75	91.08	68*ms*
BMGANet-Gated	95.20	94.85	**87.93**	91.25	90*ms*	94.31	**93.65**	89.72	**91.64**	117*ms*
BMGANet-Cross-atten	**95.21**	95.39	87.64	**91.36**	105*ms*	94.36	93.03	**90.12**	91.55	136*ms*
BMGANet-Concatenation	95.15	**95.52**	87.75	91.20	**32*ms***	94.34	92.78	89.91	91.32	**48*ms***

concatenation approach and is even slightly better in some metrics. For the same reason of high time complexity $O(d^2)$, the average detection times for BMGANet-Cross-atten for CWE-119 and CWE-399 are 105 *ms* and 136 *ms*, respectively.

The concatenation strategy directly combines the original feature vectors of function-level and token-level features without dimensionality reduction, thereby preserving the integrity of both global structural information (from function-level features) and local semantic details (from token-level features). Considering that the study only needs to fuse textual function-level (768D) and token-level (256D) features, and taking into account the low overhead advantage of concatenation strategy's linear time complexity, we ultimately adopt the concatenation strategy to fuse function-level and token-level textual features.

4.6.4. F1-scores of ablation experiments

To intuitively demonstrate the impact of different methods on the performance of the BMGANet model, Fig. 7 presents the F1-Scores from part the aforementioned ablation experiments in a bar chart.

This figure also includes the F1-Score test results of the BMGANet model without pretraining (denoted as *NoPretrain* in the figure). As observed from the figure, omitting pretraining significantly impairs BMGANet's F1-Score. Specifically, BMGANet's pretraining, encompassing dynamic masked token prediction and inter-code-line logical correlation prediction, enhances the model's semantic understanding of code slices and its ability to capture inter-line dependencies, thereby balancing precision and recall. Without pretraining, the model fails to effectively recognize vulnerability patterns, leading to more false negatives (lower recall) and false positives (lower precision), which directly reduces the F1-Score.

Fig. 7 illustrates F1-Scores of BMGANet and its ablation variants for CWE - 119 and CWE - 399. For CWE - 119, BMGANet achieves 91.2, outperforming others: NoSlice (90.88), NoNorm (89.78), Func-Only (86.29), Token-Only (89.3), Sum-Fusion (89.84), Weighted-Fusion (90.7), and NoPretrain (88.65). For CWE - 399, BMGANet hits 91.32 %, better than NoSlice (89.95), NoNorm (90.21), Func-Only (85.95), Token-Only (90.11), Sum-Fusion (90.17), Weighted-Fusion (91.08), and NoPretrain (87.94). NoPretrain has lower F1-Score across both CWEs, proving pretraining's role. Other ablations (like NoSlice, NoNorm) also show performance drops, highlighting each component's importance in BMGANet's superior vulnerability detection.

In summary, while code slicing, normalization, and pretraining collectively contribute to robust performance, the multi-granularity feature fusion architecture is the most critical component, as its removal leads to the most significant performance drop, highlighting its central role in enabling BMGANet to effectively capture vulnerability patterns.

4.7. Performance on real-world datasets

To evaluate the robustness of BMGANet in practical software development environments, we conducted experiments on two real-world datasets, Devign [12] and REVEAL [38]. These datasets are characterized by high complexity, mixed vulnerability patterns, and the presence of semantic noise, which starkly contrast with controlled synthetic datasets.

It is worth noting that the results for VulDeePecker [22], FUNDED [32], and VDoTR [30] could not be fully reproduced. This was due to the unavailability of their complete source code and insufficiently detailed training parameter configurations. Consequently, only the methodology described in the respective papers was reimplemented for these models, and training parameters were re-tuned. In all cases, the experimental results deviated significantly from those reported in the original publications. Owing to these discrepancies, the experimental evaluation and analysis in this section do not encompass these three models. As a supplementary analysis, we evaluated two additional models that follow a similar technical trajectory to this study: VulBERTa [27] and LineVul [40]. VulBERTa is designed to enhance the AI model's ability to "comprehend" code, thereby identifying underlying vulnerabilities. LineVul, on the other hand, is a line-level vulnerability detection model built upon CodeBERT.

(1). Overall performance

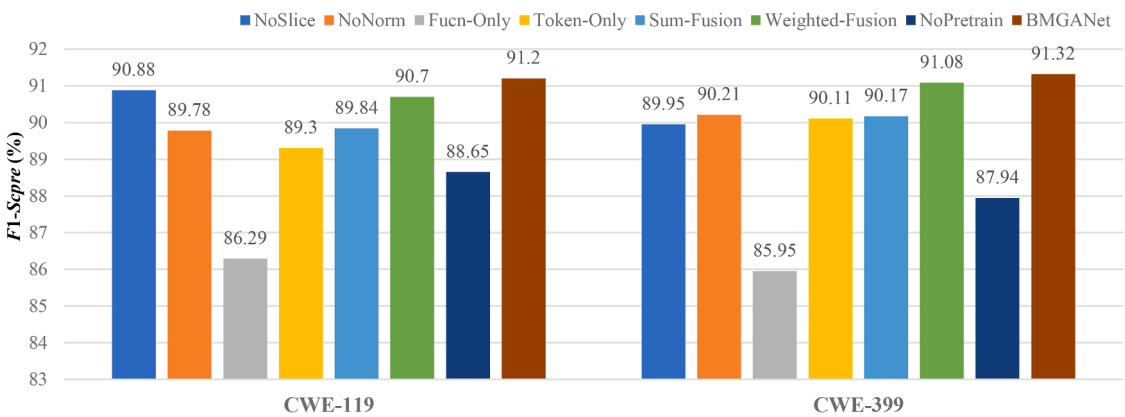

Fig. 7. F1-Score comparison of BMGANet and its ablation variants for CWE-119 and CWE-399.

Table 9 presents the comprehensive performance metrics, including accuracy, precision, recall, F1-Score, FNR, and inference time cost. The results demonstrate that BMGANet achieves the best balance between precision and recall, significantly outperforming most baseline models in F1-Score and error-related metrics. Specifically, BMGANet achieves an average of inference speed of 92.83 ms on Devign and 75.22 ms on REVEAL, making it suitable for practical integration into development pipelines.

(2). Analysis of error situations and false negatives

As shown in Table 9, BMGANet achieves a lower FNR compared to other models, demonstrating the practical robustness of the proposed approach. In real-world security auditing, false negatives (missing actual vulnerabilities) are more costly than false positives. BMGANet achieves the lowest FNR among all compared models (33.43 % on Devign and 34.03 % on REVEAL) better than μVulDeePecker (FNR 60.05 %) and SySeVR (FNR 42.15 %).

However, due to "semantic noise" inherent in real-world code, the precision of BMGANet (67.39 % on Devign) is lower than its recall. This indicates the presence of false positives of BMGANet. BMGANet intentionally optimizes for lower FNR (higher recall) to prioritize security assurance, accepting a precision trade-off to ensure no critical vulnerability is overlooked.

(3). Examination of model performance on authentic noisy code

Despite BMGANet's superiority over the baselines, a performance gap exists when comparing these results to the synthetic *N&S* dataset (where accuracy exceeds 94 %). This gap highlights the challenge posed by authentic, noisy code. Actually, the Devign and REVEAL contain complex scenarios that challenge static analysis and syntax rules:

1. *Cross-file dependencies*. Unlike synthetic datasets, real-world vulnerabilities often involve data flows cross multiple files. Our current slicing method, while effective, sometimes fails to capture the complete dependency chain when critical nodes reside in separate files.
2. *Composite logic and obfuscation*. Real-world code often contains convoluted logic and obfuscation techniques. For example, vulnerabilities requiring dynamic taint analysis or those hidden within macro definitions are difficult to detect using static syntactic rules alone.
3. *Limitations of syntax rules*. Although BMGANet significantly outperforms SySeVR (the pioneering system that introduced the four syntax rules), the complexity of many Devign and REVEAL samples exceeds the comprehensive coverage capabilities of these rules. This results in a significant decline in BMGANet's performance across all metrics compared to its results on the synthetic *N&S* dataset.

The decline in performance, particularly in comparison to VulBERTa's high accuracy (84.48 %), suggests that while pre-trained models can learn strong general representations, they may also overfit to certain syntactic patterns in large corpora, sometimes at the cost of logical precision. In contrast, BMGANet's design, which explicitly models token-level sequences and function-level semantics, provides a more interpretable and balanced detection mechanism, even if it means sacrificing a few percentage points in absolute accuracy to gain robustness against false negatives.

(4). Inference time cost analysis

BMGANet achieves average inference times of 92.83 ms on Devign and 75.22 ms on REVEAL, outperforming the CodeBERT-based models mVulSniffer, VulBERTa (MLP), and LineVul in terms of efficiency. This performance gain is attributed to our optimized feature fusion strategy (via concatenation) and pruned preprocessing (code slicing and normalization). However, due to the integration of computationally intensive components such as BERT, LSTM, and multi-head attention mechanisms, BMGANet exhibits a higher time complexity compared to μVulDeePecker, which relies on linear recursive processes, and SySeVR, which combines program slicing with shallow models (RNN/MLP).

Table 9
Vulnerability detection performance of different models on real-world datasets (%).

Models	Devign						REVEAL					
	A	P	R	F1	FNR	Time Cost (*ms*)	A	P	R	F1	FNR	Time Cost (*ms*)
μVulDeePecker	41.72	43.65	38.95	41.47	60.05	**66.74**	39.62	40.38	38.75	39.55	61.25	**55.85**
SySeVR	45.82	45.01	57.85	50.63	42.15	75.48	43.37	45.62	48.18	46.87	51.82	62.89
CodeBERT-based	56.93	51.12	50.32	50.72	49.68	144.65	61.25	48.18	41.75	54.38	44.88	112.20
mVulSniffer	61.75	57.25	54.17	55.67	45.83	119.53	58.62	51.56	54.38	52.93	45.62	96.35
VulBERTa (MLP)	**84.48**	35.18	63.48	45.27	40.34	168.29	**82.37**	43.60	43.10	43.35	56.90	122.77
LineVul	65.48	62.55	61.32	61.93	38.68	102.91	63.52	64.21	59.66	61.86	40.34	83.94
BMGANet	68.17	**67.39**	**66.57**	**66.88**	**33.43**	92.83	66.75	**67.18**	**65.97**	**66.58**	**34.03**	75.22

4.8. Case study: interpretability and localization

After BMGANet detects a vulnerability within the target code, this case study first determines the specific vulnerability type using function-level features, and subsequently identifies the high-risk code regions through a token-level attention mechanism.

(1). Vulnerability type matching based on function-level features

First, BMGANet extracts features from each sample in the accessible CWE database. Specifically, it utilizes the 768-dimensional embedding vector derived from the "*[CLS]*" token of BERT as the feature vector representing the CWE type. Through this process, the model constructs a CWE type feature vector set $\boldsymbol{P}=\{\boldsymbol{P}_0, \boldsymbol{P}_1, \ldots, \boldsymbol{P}_{n\text{-}1}\}$, where $\boldsymbol{P}_i$ represents the typical feature vector of the ith vulnerability type. Second, BMGANet generates the function-level feature vector $\boldsymbol{F}_{Func}$ for the target code snippet using the same methodology. Since both the vectors in $\boldsymbol{P}$ and $\boldsymbol{F}_{Func}$ represent the holistic semantic features of the function, the model determines the specific vulnerability category by calculating their similarity. Finally, it calculates the cosine similarity $Sim(\boldsymbol{F}_{Func}, \boldsymbol{P}_i)$ between $\boldsymbol{F}_{Func}$ and every vector $\boldsymbol{P}_i$ in the predefined set $\boldsymbol{P}$ to identify the most likely vulnerability type.

(2). Localization of suspicious code regions based on multi-head attention mechanism

As shown in Eqs. (13-14), BMGANet employs a multi-head attention mechanism (utilizing 8 heads in this study) during the token-level optimization stage. The localization of suspicious code regions is performed according to the following steps:

1. *Correlation Scoring*. Uses the $\boldsymbol{H}_D$ vector (which represents the overall semantics of the code snippet) as the ***Query***. Then calculates the correlation score $\boldsymbol{e}_t$ between this ***Query*** and the ***Key*** vectors representing the local semantics of each individual token.
2. *Attention Weight Generation*. Applies the *Softmax()* function to generate attention weights $\boldsymbol{a}_t$, quantifying the contribution of each token to the final classification decision.
3). *Localization*. The location of high-risk vulnerabilities is determined by analyzing these attention weights.
(3). A case study for CWE-119 buffer overflow vulnerability

BMGANet outputs a detection result indicating the presence of a vulnerability when analyzing the following C code snippet:

```
void vulnerable_copy(char* userinput) {
  char buffer[64];
strcpy(buffer, userinput); // dangerous call
}
```

4). *Vulnerability type determination:* First, based on the order of CWE-IDs in Table 1, BMGANet constructs the feature vector set $\boldsymbol{P}=\{\boldsymbol{P}_0, \boldsymbol{P}_1, \ldots, \boldsymbol{P}_9\}$ for the 10 vulnerability types involved in the *N&S* dataset. Second, it extracts the function-level feature $\boldsymbol{F}_{Func}$ for the aforementioned code snippet. Third, it calculates the similarity $Sim(\boldsymbol{F}_{Func}, \boldsymbol{P}_i)$ for all i values (0 through 9). Fourth, it ranks these similarity scores in descending order. For example, $Sim(\boldsymbol{F}_{Func}, \boldsymbol{P}_2)=0.93$, $Sim(\boldsymbol{F}_{Func}, \boldsymbol{P}_6)=0.21$, and $Sim(\boldsymbol{F}_{Func}, \boldsymbol{P}_7)=0.18$. According to Table 2, $\boldsymbol{P}_2$, $\boldsymbol{P}_6$, and $\boldsymbol{P}_7$ correspond to CWE-119, CWE-404, and CWE-476, respectively. Finally, the model selects the vulnerability pattern with the highest similarity as the match. In this case, since the similarity between $\boldsymbol{F}_{Func}$ and the "unsafe string operation" pattern $\boldsymbol{P}_2$ reaches 0.93, the code snippet is classified as "*CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer*".
5). *Code region localization guided by vulnerability type:* The token sequence for the example source code is as follows:

```
tokens = [
        ["void", "FUN1", "(", "char*", "VAR1", ")"],
        ["{"],
        ["char", "VAR2[VAR3]", ";"],
            ["strcpy", "(", "VAR2", ",", "VAR1", ")", ";"],
            ["}"]
]
```

By calculating and quantifying the attention weights of each token, the model generates the following attention weight matrix (where values are within the interval [0,1], and higher values indicate higher attention): attention_weights = [

```
        [0.08, 0.12, 0.05, 0.15, 0.67, 0.05],
        [0.05],
        [0.10, 0.53, 0.03],
        [0.82, 0.03, 0.71, 0.02, 0.67, 0.03, 0.03],
        [0.05]
]
```

The matrix indicates that the token associated with the attention weight of 0.82 is the high-risk code. By inversely mapping these attention weights back to the source code token sequence, BMGANet successfully localizes the vulnerable region. For intuitive analysis, visualization tools such as heatmaps can be employed to highlight these high-risk code segments.

5. Conclusion and future work

This study proposes BMGANet, a deep learning model for code vulnerability detection that integrates token-level and function-level features by leveraging PDGs and ASTs for cross-function code slicing. Combining BERT-based semantic embedding with LSTM and multi-head attention, BMGANet effectively captures both local details and global context. Enhanced by pretraining tasks that predict masked tokens and inter-code-line logical correlations, the model achieves an average accuracy of 94.48 % and F1-Score of 89.19 % on 10 CWE types, showing strong performance on synthetic and real-world datasets (Devign, REVEAL). However, several limitations point to important directions for future work. First, the current slicing and analysis are limited to single files, making the model ineffective for cross-file vulnerabilities. Future efforts will introduce cross-file program dependency analysis and integrate GNNs to model inter-file node associations, building a more comprehensive global PDG. Second, the model lacks dynamic execution context and taint propagation tracking, which limits its ability to detect vulnerabilities relying on runtime data flow, such as certain logic bugs. To address this, we will incorporate data flow and control flow analysis to trace tainted inputs across function boundaries, enhancing detection of stealthy vulnerabilities. Third, BMGANet is primarily trained on common vulnerability patterns and performs less well on rare types beyond the four dominant CWE categoriess. To improve generalization, we will explore few-shot learning and transfer learning techniques, enabling the model to learn from limited examples of novel vulnerabilities. Additionally, a fallback mechanism will be introduced to handle functions without slicing sources, thereby enhancing the model's robustness in real-world scenarios. In summary, while BMGANet achieves state-of-the-art results in static vulnerability detection, advancing cross-module dependency modeling, integrating dynamic analysis, and expanding to rare vulnerability types remain critical challenges for future research. These improvements will push the boundary of automated tools toward more comprehensive and practical security auditing in complex software systems.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Erzhou Zhu reports financial support was provided by Erzhou Zhu, Associate Professor. Erzhou Zhu reports a relationship with Natural Science Foundation of Anhui Province that includes: funding grants. Erzhou Zhu reports a relationship with University Natural Science Research Project of Anhui Province that includes: funding grants. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study was supported by the Natural Science Foundation of Anhui Province (China), (No. 2008085MF188), and the University Natural Science Research Project of Anhui Province (China) (No. KJ2021A0041).

Data availability

Data will be made available on request.

References

[1] CVE program mission. [Online] Available: https://cve.mitre.org/, Accessed in July 2025.
[2] Yu Bo, Fang Ying, Yang Qiang, Tang Yong, Liu Liu. A survey of malware behavior description and analysis. Front Inf Technol Electron Eng 2018;19:583–603. https://doi.org/10.1631/FITEE.1601745.
[3] Heiding Fredrik, Katsikeas Sotirios, Lagerstroem Robert. Research communities in cyber security vulnerability assessments: a comprehensive literature review. Comput Sci Rev 2023;48:100551. https://doi.org/10.1016/j.cosrev.2023.100551.
[4] Harzevili Nima shiri, Belle Alvine boaye, Wang Junjie, Wang Song, Jiang Zhenming, Nagappan Nachiappan. A systematic literature review on automated software vulnerability detection using machine learning. ACM Comput Surv 2025;57(3):55. https://doi.org/10.1145/3699711.
[5] Uprety Aashma, Rawat Danda B. Reinforcement Learning for IoT security: a comprehensive survey. IEEE Internet Things J 2021;8(11):8693–706. https://doi.org/10.1109/JIOT.2020.3040957.
[6] Liang Chen, Wei Qiang, Du Jiang, Wang Yisen, Jiang Zirui. Survey of source code vulnerability analysis based on deep learning. Comput Secur 2025;148:104098. https://doi.org/10.1016/j.cose.2024.104098.
[7] Yu Xiao, Lin Guancheng, Hu Xing, Keung Jacky Wai, Xia Xin. Less is more: unlocking semi-supervised deep learning for vulnerability detection. ACM Trans Softw Eng Methodol 2025;34(3):62. https://doi.org/10.1145/3699602.
[8] Wu Bolun, Zou Futai. Code vulnerability detection based on deep sequence and graph models: a survey. Secur Commun Netw 2022;2022:1176898. https://doi.org/10.1155/2022/1176898.
[9] Li Zhen, Zou Deqing, Xu Shouhuai, Jin Hai, Zhu Yawei, Chen Zhaoxuan. SySeVR: a framework for using deep learning to detect software vulnerabilities. IEEE Trans Dependable Secure Comput 2022;19(4):2244–58. https://doi.org/10.1109/TDSC.2021.3051525.
[10] He Haitao, Wang Sheng, Wang Yanmin, Liu Ke, Yu Lu. VulTR: software vulnerability detection model based on multi-layer key feature enhancement. Comput Secur 2025;148:104139. https://doi.org/10.1016/j.cose.2024.104139.
[11] Cao Sicong, Sun Xiaobing, Bo Lili, Wei Ying, Li Bin. BGNN4VD: constructing bidirectional graph neural-network for vulnerability detection. Inf Softw Technol 2021;136:106576. https://doi.org/10.1016/j.infsof.2021.106576.
[12] Yaqin Zhou, Shangqing Liu, Jing Kai Siow, Xiaoning Du, Yang Liu. Devign: effective vulnerability identification by learning comprehensive program semantics via graph Neural networks. In: Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS), Dec. 8-14, 2019, Vancouver, BC, Canada, pp.10197–10207.
[13] Rosmael Zidane Lekeufack Foulefack, Alessandro Marchetto. A rapid review on graph-based learning vulnerability detection. In: Proceedings of the 17th International Conference on the Quality of Information and Communications Technology (QUATIC), Sept. 11-13, 2024, Pisa, Italy, pp.355–372. DOI: 10.1007/978-3-031-70245-7_25.

[14] Jiang Yuan, Zhang Yujian, Su Xiaohong, Treude Christoph, Wang Tiantian. StagedVulBERT: multigranular vulnerability detection with a novel pretrained code model. IEEE Trans Softw Eng 2024;50(12):3454–71. https://doi.org/10.1109/TSE.2024.3493245.
[15] Zhu Yuhui, Lin Guanjun, Song Lipeng, Zhang Jun. The application of neural network for software vulnerability detection: a review. Neural Comput Appl 2023; 35(2):1279–301. https://doi.org/10.1007/s00521-022-08046-y.
[16] Calzavara Stefano, Conti Mauro, Focardi Riccardo, Rabitti Alvise, Tolomei Gabriele. Machine learning algorithms for robust software vulnerability detection. IEEE Secur Priv 2020;18(3):8–16. https://doi.org/10.1109/MSEC.2019.2961649.
[17] Zheng Wei, Gao Jialiang, Wu Xiaoxue, Liu Fengyu, Xun Yuxing, Liu Guoliang, Chen Xiang. The impact factors on the performance of machine learning-based vulnerability detection: a comparative study. J Syst Softw 2020;168:110659. https://doi.org/10.1016/j.jss.2020.110659.
[18] Supriya Shakya, Arnab Mukherjee, Raju Halder, Abyayananda Maiti, Amrita Chaturvedi. SmartMixModel: machine learning-based vulnerability detection of solidity smart contracts. In: Proceedings of the IEEE 2022 international conference on blockchain (blockchain), Aug. 22, 2022, Espoo, Finland, pp.37–44. DOI: 10.1109/Blockchain55522.2022.00016.
[19] Benjamin Steenhoek, Md Mahbubur Rahman, Richard Jiles, Wei Le. An empirical study of deep learning models for vulnerability detection. In: Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE), May 14-20, 2023, Melbourne, Australia, pp.2237–2248. DOI: 10.1109/ICSE48619.2023.00188.
[20] Wan Yao, Bi Zhangqian, He Yang, Zhang Jianguo, Zhang Hongyu, Sui Yulei, Xu Guandong, Jin Hai, Yu Philip S. Deep learning for code intelligence: survey, benchmark and toolkit. ACM Comput Surv 2024;56(12):309. https://doi.org/10.1145/3664597.
[21] Li Zhen, Zou Deqing, Xu Shouhuai, Chen Zhaoxuan, Zhu Yawei, Jin Hai. VulDeeLocator: a deep learning-based fine-grained vulnerability detector. IEEE Trans Dependable Secure Comput 2022;19(4):2821–37. https://doi.org/10.1109/TDSC.2021.3076142.
[22] Zhen Li, Deqing Zou, Shouhuai Xu, Xinyu Ou, Hai Jin, Sujuan Wang, Zhijun Deng, Yuyi Zhong. VulDeePecker: a deep learning-based system for vulnerability detection. In: Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS), Feb. 18-21, 2018, San Diego, California, USA. DOI: 10.14722/ndss.2018.23158.
[23] Zou Deqing, Wang Sujuan, Xu Shouhuai, Li Zhen, Jin Hai. μVulDeePecker: a deep learning-based system for multiclass vulnerability detection. IEEE Trans Dependable Secure Comput 2021;18(5):2224–36. https://doi.org/10.1109/TDSC.2019.2942930.
[24] Software assurance reference dataset. [Online] Available: https://samate.nist.gov/SRD/index.php/, Accessed in July 2025.
[25] National vulnirability database. [Online] Available: https://nvd.nist.gov/, Accessed in July 2025.
[26] Tao Wenxin, Su Xiaohong, Ke Yekun, Han Yi, Zheng Yu, Wei Hongwei. Transformer-based statement level vulnerability detection by cross-modal fine-grained features capture. Knowl Based Syst 2025;316:113341. https://doi.org/10.1016/j.knosys.2025.113341.
[27] Hazim Hanif, Sergio Maffeis. VulBERTa: Simplified source code pre-training for vulnerability detection. In: Proceedings of the International Joint Conference on Neural Networks (IJCNN), July 18-23, 2022, Padua, Italy, pp.1–8. DOI: 10.1109/IJCNN55064.2022.9892280.
[28] Chandra Thapa, Seung Ick Jang, Muhammad Ejaz Ahmed, Seyit Camtepe, Josef Pieprzyk, Surya Nepal. Transformer-based language models for software vulnerability detection. In: Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC), Dec. 5-9, 2022, Austin, TX, USA, pp.481–496. DOI: 10.1145/3564625.3567985.
[29] Cheng Xiao, Wang Haoyu, Hua Jiayi, Xu Guoai, Sui Yulei. DeepWukong: statically detecting software vulnerabilities using deep graph neural network. ACM Trans Softw Eng Methodol 2021;30(3):38. https://doi.org/10.1145/3436877.
[30] Fan Yuanhai, Wan Chuanhao, Fu Cai, Han Lansheng, Xu Hao. VDoTR: vulnerability detection based on tensor representation of comprehensive code graphs. Comput Secur 2023;130:103247. https://doi.org/10.1016/j.cose.2023.103247.
[31] Shao Miaomiao, Ding Yuxin, Cao Jing, Li Yilin. GraphFVD: property graph-based fine-grained vulnerability detection. Comput Secur 2025;151:104350. https://doi.org/10.1016/j.cose.2025.104350.
[32] Wang Huanting, Ye Guixin, Tang Zhanyong, Tan Shin Hwei, Huang Songfang, Fang Dingyi, Feng Yansong, Bian Lizhong, Wang Zheng. Combining graph-based learning with automated data collection for code vulnerability detection. IEEE Trans Inf Forens Secur 2021;16:1943–58. https://doi.org/10.1109/TIFS.2020.3044773.
[33] Cui Lei, Hao Zhiyu, Jiao Yang, Fei Haiqiang, Yun Xiaochun. VulDetector: detecting vulnerabilities using weighted feature graph comparison. IEEE Trans Inf Forens Secur 2021;16:2004–17. https://doi.org/10.1109/TIFS.2020.3047756.
[34] Huang Yuanming, He Mingshu, Wang Xiaojuan, Zhang Jie. HeVulD: a static vulnerability detection method using heterogeneous graph code representation. IEEE Trans Inf Forens Secur 2024;19:9129–44. https://doi.org/10.1109/TIFS.2024.3457162.
[35] Zheng Weining, Su Xiaohong, Wei Hongwei, Tao Wenxin. SVulDetector: vulnerability detection based on similarity using tree-based attention and weighted graph embedding mechanisms. Comput Secur 2024;144:103930. https://doi.org/10.1016/j.cose.2024.103930.
[36] Miaomiao Shao, Yuxin Ding. FVD-DPM: fine-grained vulnerability detection via conditional diffusion probabilistic models. In: Proceedings of the 33rd USENIX security symposium (USENIX security), Aug. 14-16, 2024, Philadelphia, PA, USA, pp.7375–7392.
[37] Bozhi Wu, Shangqing Liu, Yang Xiao, Zhiming Li, Jun Sun, Shang-Wei Lin. Learning program semantics for vulnerability detection via vulnerability-specific inter-procedural slicing. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Dec. 3-9, 2023, San Francisco, CA, USA, pp.1371–1383. DOI: 10.1145/3611643.3616351.
[38] ReVeal. [Online] Available: https://github.com/VulDetProject/ReVeal, Accessed in July 2025.
[39] Zhang Xuejun, Zhang Fenghe, Gai Jiyang, Du Xiaogang, Zhou Wenjie, Cai Teli, Zhao Bo. mVulSniffer: a multi-type source code vulnerability sniffer method. Tongxin Xuebao/J Commun 2023;44(9):149–60. https://doi.org/10.11959/j.issn.1000-436x.2023184.
[40] Michael Fu, Chakkrit Tantithamthavorn. LineVul: a transformer-based line-level vulnerability prediction. In: Proceedings of the 19th IEEE/ACM international conference on Mining Software Repositories (MSR), May 23-24, 2022, Pittsburgh, PA, USA, pp.608–620. DOI: 10.1145/3524842.3528452.